

MinMandate: Private Task-Scoped Payment Authorization for Adaptive Agent Workflows

Ge Gao, Haining Yu*, Zhichao Liu, Dongyang Zhan, Yuanxiao Zhu, Zhongyun Hua

Harbin Institute of Technology, China

Abstract

Autonomous agents are increasingly used to plan and execute paid workflows on behalf of users. Existing agentic-payment frameworks support this delegation through merchant-admission authorization credentials but require the user to specify merchants before execution. However, complex paid workflows often span multiple services and merchants, and agents may choose among them based on intermediate results. This creates two limitations: (1) requiring the user to choose each merchant in advance either limits the agent’s adaptability or forces the user back into the loop; and (2) reusing a stable identifier across merchants lets observers link separate paid calls and infer the user’s broader intent. To address these limitations, we introduce MinMandate, which grants adaptive merchant selection within user-approved task bounds and derives fresh per-call payment views without introducing a stable cross-merchant identifier. Extensive experiments on AgentDojo tasks demonstrate that, when 50% of merchants are unavailable, MinMandate improves task success by 32.7 percentage points on average across four tested planners compared with an AP2 baseline that preauthorizes one merchant per service class. Reintroducing a reusable public payment-layer handle in the Stable Handle ablation raises attacker task-recovery success by 27.4 percentage points on average, isolating the privacy cost of a stable join handle. The code is available at <https://github.com/Zora-G/minmandate>.

Introduction

Autonomous agents are rapidly becoming active economic participants, driving the development of agentic commerce. Commercial services like Clay and Attio already demonstrate strong demand for these paid workflows (Clay 2026a,b; Attio 2026). Standards like the Model Context Protocol (MCP) already provide an interface for agents to call external tools (Anthropic 2024; Model Context Protocol 2025). However, they leave a payment gap: when these tools are paid services, agents also need a way to authorize payments. Traditional payment systems are designed for humans, relying on manual checkout steps and online approvals that interrupt autonomous execution. Existing agentic-payment frameworks such as the Agentic Payment Protocol (AP2) support this delegation through merchant-specific authorization mandates that allow agents to initiate payments on the user’s behalf for merchants specified before execution (Google 2026).

AP2 closes this basic payment gap, but its authorization credentials are a poor fit for complex paid workflows. First, such workflows often span multiple services and merchants, and the agent may not know which merchant it needs until execution. Requiring the user to choose each merchant in advance either limits the agent’s adaptability or forces the user back into the loop. If a specified merchant is unavailable, exceeds the budget, or returns poor results, the agent cannot switch to another merchant that still satisfies the user’s requirements without a new user-signed credential, and the task can fail. Pre-authorizing more merchants can reduce this failure risk, but it shifts the burden back to the user: the user must predict and sign every merchant that the agent may need as workflows span more services and merchants. Second, AP2 records expose a stable holder-key identifier, such as `open_checkout_mandate.cnf.kid`, across merchants. An observer that sees the same value in separate paid calls can join those calls and infer the user’s broader intent. Broadening the authorization to support flexible merchant choice creates another leak: it can reveal task context that a single verifier does not need. For example, in a product due-diligence task, a patent search service only needs the patent-search request and the payment limit for that call. It should not learn that the same task also authorizes vulnerability scanning of the product, or that the user reserved a larger budget for the full workflow. The goal is not to remove every call identifier. A paid call still needs a fresh per-call binding: this authorization applies to this request and this quote. Without that binding, an attacker could attach the same authorization evidence to a different request or quote, or replay it for a second spend. Thus the merchant acknowledgement must sign a fresh per-call binding, while the protocol must avoid any stable identifier reused across merchants.

In this paper, we introduce **MinMandate**, a privacy-preserving agentic payment framework for adaptive multi-merchant workflows. Our design follows two principles. First, *task-scoped payment authority* grants payment authority for a general task scope rather than specific merchants, so the agent can choose among policy-compliant merchants at runtime while staying within the user’s approved service category, budget, and spending limits. Second, *unlinkable per-call views* derive fresh service and spend views that contain credential proofs for every paid call, without introducing a stable payment identifier into external transcripts. Guided by these

*Corresponding author: yuhaining@hit.edu.cn.

principles, we formalize a privacy model for agentic payment workflows and implement MinMandate. Our evaluation on AgentDojo tasks shows that payment-layer privacy can be added without further reducing task success beyond what the approved policy permits. We summarize our contributions as follows:

- ◊ We identify and quantify a payment-layer linkage risk in agentic workflows: a stable identifier reused across merchants lets attackers link separate paid calls and infer the user’s high-level task intent.
- ◊ We design two mechanisms: task-scoped payment authority provides adaptive payment authority without fixing merchants in advance; unlinkable per-call views derive fresh service and spend views per call, without introducing a stable payment identifier.
- ◊ We propose MinMandate, a privacy-preserving agentic payment framework for adaptive, multi-merchant paid workflows. It enables dynamic merchant selection without requiring users to approve each payment online.
- ◊ We conduct extensive experiments showing that MinMandate improves adaptability over baselines, maintains higher task success when some merchants are unavailable, and lowers attacker task-recovery success.

Background and Problem Setting

Related Work

Existing agentic payment protocols each address a complementary function: AP2 provides mandates, x402 and MPP carry HTTP-native payments, and Stripe Shared Payment Tokens restrict a payment method by seller, amount, and expiry (Google 2026; Parikh and Surapaneni 2025; Reppel et al. 2025; Weinstein and Kaliski 2026; Stripe 2026b,a). AP2’s recurring SD-JWT VC mandates allow an agent to specify constraints such as merchant identity or line items (Google 2026; Fett, Yasuda, and Campbell 2025). However, these values remain visible to all verifiers, and AP2 defines no per-verifier minimum disclosure profile; reusing a credential retains its issuer-signed JWT, which links presentations across calls (Google 2026; Fett, Yasuda, and Campbell 2025). MinMandate instead derives fresh service and spend views that prove merchant admission, price bounds, and capacity while hiding unused scope. We call this *payment-layer workflow privacy*: verifier-local disclosure and cross-call unlinkability under declared leakage.

ToolEmu and AgentDojo evaluate tool-agent risks. MCP attack studies and PrivacyLens, SAPA, and Portcullis examine tool- and application-layer security and privacy (Ruan et al. 2024; Debenedetti et al. 2024; Wang et al. 2026b,a; Shao et al. 2024; Lin et al. 2026; Zhan et al. 2025). These works do not model multi-party payment transcripts. Agent-payment analyses study metadata leakage, binding and replay, free-riding, and exchange atomicity (Stantchev 2026; Li, Wang, and Wang 2026; Ling et al. 2026; Li et al. 2026). This line of work is complementary to our disclosure and linkage focus.

System and Security Model

System model. User U approves a mandate that specifies the permitted service classes and merchant-selection con-

straints, budget, spend slots, and expiry. **Agent A** receives task-scoped payment authority, obtains each quote, and derives a service view and a spend view. **Merchant M_i** verifies its service view, returns a merchant acknowledgement, and releases its output after receiving the settlement authorization returned by $\mathcal{W}_{\text{red}}$. **Wallet W** uses $\mathcal{W}_{\text{iss}}$ to verify the approval and funding reserve and issue an authorization credential, and uses $\mathcal{W}_{\text{red}}$ to verify both views and the acknowledgement, consume spend evidence, and return a receipt.

Threat model. For soundness, the agent A and any subset of policy-compliant merchants are fully malicious and may collude. They adaptively choose requests, quotes, merchant acknowledgements, and redemption inputs, and attempt unauthorized redemptions by replaying, interleaving, or racing calls. The user U , wallet W , and setup procedure are honest. For external-payment-layer privacy, U , A , and W are honest. The adversary consists of colluding policy-compliant merchants and external payment-layer observers that pool external transcripts to link calls. We define unlinkability after fixing the information already revealed by the environment: task decomposition, selected merchants, service inputs, prices, timing and ordering, merchant and payee choices, payment metadata, and network or public-log data. Under this fixed leakage, the payment transcript should not add a stable cross-merchant identifier. Wallet-side issuance reveals the complete signed vector, including serial and funding-tag seeds, to the honest wallet, so honest-but-curious wallet privacy is outside our main model. It can be strengthened with the optional blind-issuance extension and non-colluding issuance and redemption services. Formal security definitions, games, and proofs appear in Appendix C.

Cryptographic Primitives

Notation. All hash inputs use injective canonical encodings Enc_{can} . We use three hash names with fixed roles. $\mathbf{H}_{\text{can}} : \{0, 1\}^* \rightarrow \mathbb{Z}_p$ is the canonical scalar digest for encoded strings. $\mathbf{H}_{\text{bind}}$ is a domain-separated scalar digest for binding encoded tuples. $\mathbf{H}_{\text{grp}} : \{0, 1\}^* \rightarrow \mathbb{G}_1$ maps encoded strings to group points. **Sig** = (**KeyGen**, **Sign**, **Verify**) denotes a standard EUF-CMA signature scheme (Goldwasser, Micali, and Rivest 1988). Pedersen commitments (Pedersen 1992) use the interface **ComScheme** = (**Setup**, **Commit**, **Open**, **Verify**). Its commitment maps Com_{vec} (vector), Com (single scalar), and Com_{link} (two-coordinate instance) are computationally binding and hiding in the standard sense used below.

Non-Interactive Zero-Knowledge (NIZK) Proof. A NIZK proof (Fiat and Shamir 1987; Groth and Sahai 2008) convinces a verifier that a statement holds without revealing the witness. The proof system for relation $\mathcal{R}$ is **NIZK** = (**Setup**, **Prove**, **Verify**), where $\pi \leftarrow \text{Prove}(x, w)$ generates a proof for statement x using witness w , and $\text{Verify}(x, \pi) \in \{0, 1\}$ checks it. We require completeness (every valid witness yields an accepting proof), knowledge soundness (an accepting proof guarantees knowledge of a valid witness), and zero knowledge (the proof leaks nothing beyond the truth of the statement).

BBS Credentials. A BBS credential (Looker et al. 2026;

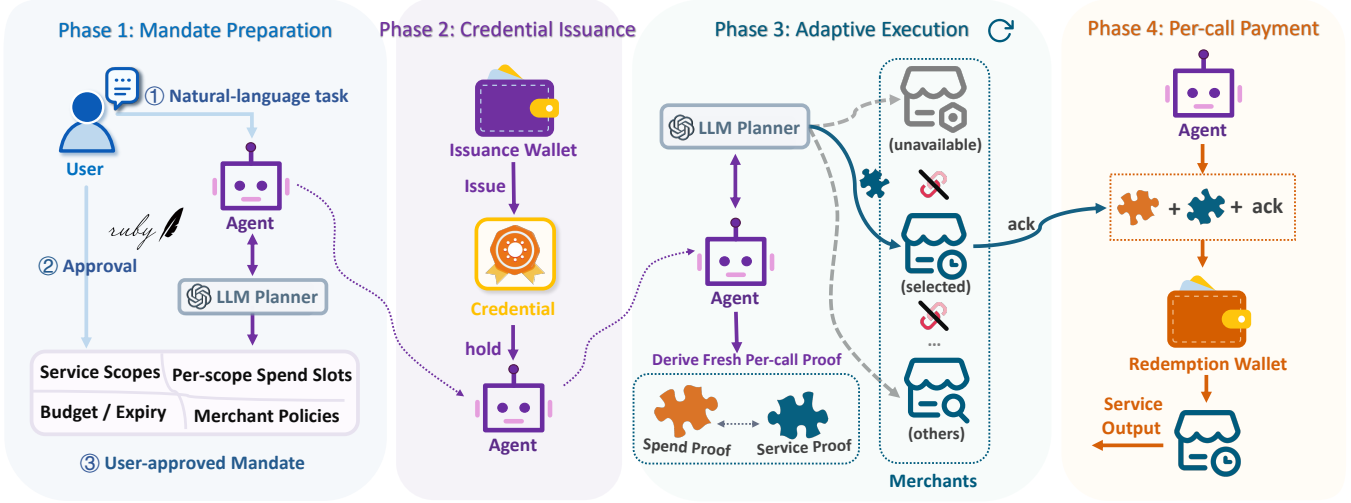


Figure 1: MinMandate workflow. The user signs one mandate, and the issuance wallet issues an authorization credential. Each paid call derives fresh service and spend views; the redemption wallet accepts only if both views and the merchant acknowledgement verify against the same per-call consistency binding.

World Wide Web Consortium 2026) is a pairing-based signature over message vectors that supports selective disclosure: presentations reveal selected signed attributes and hide the rest. The credential interface is **BBS = (Setup, KeyGen, Issue, Present, Verify)**. The scheme operates over order- p type-3 groups $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T)$ with pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$. Signed attributes are scalars in $\mathbb{Z}_p$; signatures are pairs (A, e) with $A \in \mathbb{G}_1$ and $e \in \mathbb{Z}_p$. $\text{Setup}(1^\lambda)$ outputs public parameters pp_{bbs} with authenticated generators $g_1, \{h_j\} \subset \mathbb{G}_1$. KeyGen outputs the issuer key pair (isk, ipk) . $\text{Issue}(\text{isk}, y)$ signs a message vector y and returns a credential. $\text{Present}(\text{cred}; x, w)$ derives a selective-disclosure proof for public statement x and witness w ; x records the disclosed coordinates and any auxiliary relation. $\text{Verify}(\text{ipk}, x, \pi)$ checks the presentation. We require adaptive chosen-message unforgeability, presentation proof-of-knowledge soundness, and zero knowledge.

MinMandate

Workflow of MinMandate

MinMandate combines task-scoped payment authority and unlinkable per-call views (Figure 1). Together, these mechanisms enforce task-level scope and spending limits without introducing a stable payment identifier into external transcripts. We summarize the workflow below and then expand the two core mechanisms: task-scoped payment authority and unlinkable per-call views. The per-call consistency binding is part of unlinkable per-call views.

Phase 1: Mandate Preparation. Before execution, the LLM-based planner derives a non-binding tool-call scope sketch from the natural language task request, the public tool catalog, and the user policy. This sketch is only an authorization proposal. It estimates the required service classes, budget, and possible substitute merchants, but it does not fix final merchants, dictate a call sequence, or authorize payment. The

agent maps this sketch into a typed mandate m specifying the permitted service classes, merchant-selection constraints, total budget, single-use spending slots, and an expiry. The user reviews and signs m . A later runtime choice is valid only if it remains inside this signed mandate. Because the mandate specifies merchant-selection constraints instead of merchant identities, m can cover a class of policy-compliant merchants without binding each slot to one merchant ID.

Phase 2: Credential Issuance. After the user signs the mandate, the agent interacts with the issuance wallet $\mathcal{W}_{\text{iss}}$ to execute the issuance protocol. The wallet verifies the user’s signature, funding status, and mandate well-formedness. It then reserves the approved budget and issues an authorization credential Cred over the mandate fields and agent-sampled local secrets z , forming the agent-held credential state (Cred, z) . This credential cryptographically binds the mandate’s merchant-selection constraints, service classes, expiry, and total budget to scoped, single-use spending slots. It does not let the agent change m ; it only lets the agent prove call-local facts inside m .

Phase 3: Adaptive Execution. After issuance, the planner initializes execution state from the task context, the scope sketch, and the signed mandate, then advances the task adaptively. Free tool calls follow the standard path. For paid calls, the planner may reorder calls or switch among policy-compliant merchants at runtime. For each paid call, it selects a candidate merchant, obtains a runtime quote, and checks the quote and selected slot against the service class, merchant-selection constraints, budget, and expiry of m . If the quote is covered, the agent derives a service view and a spend view from the agent-held credential state. The service view shows that the request belongs to an approved service class and that the selected merchant is policy-compliant. The spend view shows the amount, payee, selected slots, and fresh spend evidence needed for redemption. Both views carry the same per-call

<pre> {"UAA": { "budget": "USD 3.45", "expiry": "2026-12-31", "scopes": [{ "class": "web_search", "policy": "admit:web", "slots": ["k=1: USD 0.50"] }, { "class": "data_lookup", "policy": "admit:data", "slots": ["k=2: USD 1.50"] }], "signature": "Sig_U(m)" } </pre>	<pre> {"V_i^M": { "issuer_auth": "Auth_i", "anchor": "C_i = (R_i, L_i)", "request_digest": "d_i^svc", "class": "web_search", "merchant": "m2", "quote": { "amount": "USD 0.50", "payee": "m2" }, "merchant_cert": "Cert_m2", "proof": "pi_i^M", "binding": "Bind_i" } </pre>	<pre> {"V_i^R": { "issuer_auth": "Auth_i", "anchor": "C_i = (R_i, L_i)", "payment": { "merchant": "m2", "amount": "USD 0.50", "payee": "m2" }, "selected_slots": "k = 1", "serials": "N_{i,1}", "funding_tags": "T_{i,1}^F", "proof": "pi_i^R", "binding": "Bind_i" } </pre>
--	--	--

Figure 2: Canonical signed mandate, service view, and spend view. The signed mandate contains the task-level payment policy; each paid call derives call-local views sharing the per-call consistency binding Bind_i .

consistency binding, which ties the request, quote, policy-compliant merchant, payee, and selected spend evidence to one paid call. If a quote is outside m or a candidate merchant cannot complete the call, the agent records a structured denial in the planner state. The planner can then try another policy-compliant merchant within the same mandate and aborts only after bounded replanning fails. This ties task success to approved scope, not the first quoted merchant.

Phase 4: Per-call Payment. The merchant verifies the service view and returns a merchant acknowledgement that signs the per-call consistency binding. The redemption wallet then verifies proof-and-binding consistency, validates the merchant acknowledgement, checks freshness, and atomically consumes the spend evidence before returning a settlement authorization. Thus, the merchant checks service authority and the wallet checks spendability without either verifier receiving the full mandate. The merchant releases the service result only upon receiving this authorization. If any check fails, the agent may select another policy-compliant merchant within the authorization credential's bounds, but cannot increase the budget or provision new spending slots. The complete execution loop is therefore fail-closed at the payment layer. The complete workflow algorithm, credential construction, and role-specific proof relations appear in Appendices A.1 to A.3.

Task-scoped payment authority

Task-scoped payment authority compiles the planner's scope sketch, task context, and the user policy into a typed mandate

$$\begin{aligned}
 m &= (\{\text{Scope}_\ell\}_{\ell=1}^L, B, \text{expiry}), \\
 \text{Scope}_\ell &= (\gamma_\ell, \varphi_\ell, \{(k, c_{\ell,k})\}_{k \in \mathcal{K}_\ell}).
 \end{aligned} \tag{1}$$

Each Scope_ℓ authorizes one service class under a typed merchant-selection predicate and a finite set of single-use capacities. The user signs the canonical mandate encoding, producing $\text{UAA} = (m, \sigma_U)$, so any merchant satisfying φ_ℓ may be substituted without reapproval. Changing $\gamma_\ell, \varphi_\ell$, the budget, capacity, or expiry requires a newly signed mandate. Here φ_ℓ is evaluated over authenticated merchant attributes under protocol-fixed fields, operators, normalization, and

rejection rules. Amounts are nonnegative integers in the mandate asset's minor unit, and verification rejects asset mismatches, noncanonical encodings, overflow, and rounded values. Figure 2 (left) shows the signed mandate.

Holding UAA, the agent and issuance wallet $\mathcal{W}_{\text{iss}}$ run the BBS issuance protocol over the message vector $y = (\text{H}_{\text{can}}(\text{UAA}), \{\text{Scope}_\ell\}, B, \text{expiry}, s_N, s_F)$, gated by the capacity invariant

$$\sum_{\ell=1}^L \sum_{k \in \mathcal{K}_\ell} c_{\ell,k} \leq B, \quad c_{\ell,k} > 0 \quad (\ell \in [L], k \in \mathcal{K}_\ell). \tag{2}$$

Let $(\text{isk}_{\text{iss}}, \text{ipk}_{\text{iss}})$ be the issuance wallet's BBS issuer key pair. If the invariant holds, the issuance wallet reserves B and signs y :

$$\text{Cred} \leftarrow \text{BBS.Issue}(\text{isk}_{\text{iss}}, y). \tag{3}$$

The agent retains the agent-held credential state (Cred, z) , where $z = (s_N, s_F)$ are hidden seeds for deriving one-time spend evidence per call.

Unlinkable per-call views

Unlinkable per-call views derive a service view and a spend view from the agent-held credential state (Cred, z) for each paid call. It consists of call-local view derivation and the consistency binding that lets the merchant and wallet verify the same request and quote without receiving the same information. The merchant returns a runtime quote q_i . As in Figure 2, the quote exposes the amount and payee; internally it carries the service class, merchant identity, and a quote nonce for replay protection. The agent selects an authorized scope ℓ_i and slot set $\mathcal{S}_i$, then computes the service digest $d_i^{\text{svc}} = \text{H}_{\text{can}}(\text{Enc}_{\text{can}}(\text{req}_i))$. It samples fresh randomness η_i, ρ_i and produces call-local commitments

$$\begin{aligned}
 R_i &\leftarrow \text{Com}_{\text{vec}}(d_i^{\text{svc}}, q_i, \text{H}_{\text{can}}(\text{UAA}), \ell_i; \eta_i), \\
 L_i &\leftarrow \text{Com}_{\text{link}}(\text{H}_{\text{can}}(\text{UAA}), \text{H}_{\text{can}}(\text{Scope}_{\ell_i}); \rho_i).
 \end{aligned} \tag{4}$$

The anchor $C_i = (R_i, L_i)$ hides the approval digest and scope. For each selected slot $k \in \mathcal{S}_i$, the agent derives one

serial and one funding tag from the hidden seed coordinates $s_{\ell_i}^N, s_{\ell_i}^F \in z$:

$$\begin{aligned} N_{\ell_i, k} &= s_{\ell_i}^N H_{\text{grp}}(\text{Enc}_{\text{can}}(\ell_i, k)), \\ T_{\ell_i, k}^F &= s_{\ell_i}^F H_{\text{grp}}(\text{Enc}_{\text{can}}(\ell_i, k)), \quad k \in \mathcal{S}_i. \end{aligned} \quad (5)$$

The wallet uses the serial to reject double spending of the same slot. The funding tag links that serial to a funded slot. The proof π_i^P later shows that both disclosed values were derived from the hidden seed coordinates and the selected slot, without revealing those seeds. These values populate the spend-evidence fields of $\mathcal{V}_i^P$. The service view $\mathcal{V}_i^S$ carries the service class, runtime merchant, quote, authenticated admission record, and proof π_i^S that shows credential possession, digest opening, class coverage, and $\mu_i \models \varphi_{\ell_i}$. The spend view $\mathcal{V}_i^P$ carries the payment projection, selected slots, serials, funding tags, and proof π_i^P for the payment-side relation to C_i . The proofs π_i^S and π_i^P , contained in the two views, are derived by independent calls to BBS.Present:

$$\begin{aligned} \pi_i^S &\leftarrow \text{BBS.Present}(\text{Cred}; x_i^S, w_i^S), \\ \pi_i^P &\leftarrow \text{BBS.Present}(\text{Cred}; x_i^P, w_i^P). \end{aligned} \quad (6)$$

Here x_i^S, x_i^P are public statements and w_i^S, w_i^P the witness openings. The two public statements share only the issuer authorization, anchor, and per-call consistency binding. The merchant does not receive π_i^P , selected slots, serials, funding tags, or the task budget. The wallet receives both statements, but not req_i .

Per-call consistency binding. Per-call consistency binding ties each payment to one specific request, quote, and policy-compliant merchant. It is the `binding` field shared by the two views in Figure 2. It ensures $\mathcal{V}_i^S$ and $\mathcal{V}_i^P$ describe the same call. MinMandate does not introduce a stable payment identifier into external transcripts.

Let $\mathcal{V}_i^{S,0}$ and $\mathcal{V}_i^{P,0}$ be the views before the binding field is inserted. The binding is

$$\begin{aligned} d_i^S &= H_{\text{can}}(\text{Enc}_{\text{can}}(\mathcal{V}_i^{S,0})), \\ d_i^P &= H_{\text{can}}(\text{Enc}_{\text{can}}(\mathcal{V}_i^{P,0})), \\ \text{Bind}_i &= H_{\text{bind}}(C_i, d_i^{\text{svc}}, d_i^S, d_i^P). \end{aligned} \quad (7)$$

Here d_i^S and d_i^P are the digests of the two views without their binding fields. They cover the request digest, quote, merchant, payee, selected slots, serials, funding tags, and both proofs. Fresh anchor and proof randomness make Bind_i call-local. The merchant verifies issuer authorization, the certificate, π_i^S , and Bind_i . On success it returns the merchant acknowledgement

$$\text{ack}_i \leftarrow \text{Sign}(sk_{\mu_i}, \text{Enc}_{\text{can}}(R_i, d_i^{\text{svc}}, \text{Bind}_i)). \quad (8)$$

The merchant acknowledgement signs the per-call consistency binding; it does not certify price fairness, service correctness, or wallet redemption. The agent submits $(\mathcal{V}_i^S, \mathcal{V}_i^P, \text{ack}_i)$ to $\mathcal{W}_{\text{red}}$. The wallet verifies both proofs and recomputes Bind_i , thereby checking the request/quote/merchant/spend

correspondence without field equality. It then verifies ack_i on Bind_i , expiry, selected-slot distinctness and capacity, and serial/funding-tag consistency. If any check fails, wallet state is unchanged and no settlement authorization is issued. If every check passes, the wallet atomically marks fresh serials as spent, stores the accepted call digest, and returns a settlement authorization and receipt. An exact retry returns the stored receipt; modified retries or concurrent serial reuse cannot be freshly accepted.

Let $\mathcal{I}_{\text{acc}}$ be the set of calls for which the wallet freshly accepts a redemption under one credential. Because these calls use pairwise disjoint signed slots,

$$\sum_{i \in \mathcal{I}_{\text{acc}}} a_i \leq \sum_{i \in \mathcal{I}_{\text{acc}}} \sum_{k \in \mathcal{S}_i} c_{\ell_i, k} \leq \sum_{\ell=1}^L \sum_{k \in \mathcal{K}_{\ell}} c_{\ell, k} \leq B. \quad (9)$$

where a_i is the quoted amount. Thus, per-call consistency binding guarantees request–payment consistency, bounded spending, and resistance to replay and cross-call splicing under the stated role separation and freshness checks.

Evaluation

Benchmark Inputs. The **Utility Benchmark** uses 96 pre-frozen AgentDojo tasks (Debenedetti et al. 2024) with four planners: Llama 3.1 8B (L8) (Grattafiori et al. 2024), Qwen 2.5 14B (Q14) and Qwen 2.5 32B (Q32) (Yang et al. 2024), and GPT-OSS 120B (OpenAI 2025). It tests task success under merchant unavailability. The **Privacy Benchmark** uses a separate 96-task Q32 (Yang et al. 2024) real-wire task-recovery cohort with human review, Llama 3.3 70B (Meta AI 2024), BGE-base-en-v1.5 embedding similarity (Xiao et al. 2023), and MiniLM NLI cross-encoder (Sentence Transformers 2024) evaluators for cross-merchant task recovery from payment-layer records. The **Overhead Benchmark** replays a controlled four-call payment trace with fixed requests, service outputs, quotes, and protocol conditions to measure runtime and communication without planner variance. It uses a Python replay driver and a persistent Rust backend whose BLS12-381 BBS-style credential and proof path uses `halo2curves`, `ff`, `group`, and `pairing`; JSONL transport uses `serde_json`. We ran it on an Intel Xeon Gold 6133 server with one pinned CPU core and a single execution lane. For replayed task benchmarks, planner inputs, service outputs, quotes, and availability are frozen across conditions. Utility results are paired by task and planner condition; the task is the statistical unit. The three benchmarks use separate inputs and metrics. Full utility and privacy evaluation procedures appear in Appendices E.1 and E.2.

Metrics. We evaluate MinMandate along four axes. (1) **Task Utility:** task success, authorization failure, market failure, and fallback success under controlled merchant unavailability. (2) **Payment-layer Privacy:** task-recovery attack success, defined as semantic recovery of the user’s initial task from payment-layer records. (3) **Robustness:** parser, state, and cryptographic rejection checks. (4) **Runtime and Communication Overhead:** local runtime and serialized application-layer communication for setup and paid calls.

Baselines and Controls. The main comparison baseline is AP2- k , which fixes the top k merchants at authorization

Planner	Block	Condition	Task success (%) $\uparrow$ under p_{unavail}				Merchant adaptability			
			0	0.1	0.25	0.5	vs. Native at $p = .25$ (pp)	Authorization fail. (pp) $\downarrow$	Market fail. (pp) $\downarrow$	Fallback succ. (%) $\uparrow$
L8	AP2- k	AP2-1	13.9	11.5	8.7	5.9	-4.9	41.6	15.6	1.6
		AP2-2	13.9	13.2	12.5	8.3	-1.0	20.8	15.6	38.1
		AP2-3	13.9	13.9	13.5	11.5	+0.0	2.6	15.6	52.4
	Controls / Ours	Native	13.9	13.9	13.5	11.8	+0.0	0.0	15.6	54.0
		Policy-only	13.9	13.9	13.5	11.8	+0.0	0.0	15.6	54.0
		MinMandate	13.9	13.9	13.5	11.8	+0.0	0.0	15.6	54.0
Q14	AP2- k	AP2-1	77.1	67.4	49.7	29.9	-32.6	49.3	10.8	12.3
		AP2-2	77.1	74.3	66.7	51.4	-15.6	22.8	10.8	58.0
		AP2-3	77.1	77.1	75.7	62.5	-6.6	6.7	10.8	80.0
	Controls / Ours	Native	83.7	83.7	82.3	73.6	+0.0	0.0	10.8	90.2
		Policy-only	77.1	77.1	75.7	67.0	-6.6	0.0	10.8	83.3
		MinMandate	77.1	77.1	75.7	67.0	-6.6	0.0	10.8	83.3
Q32	AP2- k	AP2-1	78.5	68.1	50.7	29.2	-36.5	51.1	11.0	12.7
		AP2-2	78.5	75.3	67.4	51.0	-19.8	24.3	11.0	57.7
		AP2-3	78.5	78.5	77.1	63.9	-10.1	6.6	11.0	81.0
	Controls / Ours	Native	88.5	88.5	87.2	78.5	+0.0	0.0	11.0	92.2
		Policy-only	78.5	78.5	77.4	68.8	-9.7	0.0	11.0	85.7
		MinMandate	78.5	78.5	77.4	68.8	-9.7	0.0	11.0	85.7
GPT-OSS	AP2- k	AP2-1	94.1	81.2	59.7	35.1	-33.0	50.7	11.5	12.7
		AP2-2	94.1	90.6	81.6	60.8	-11.1	24.0	11.5	65.1
		AP2-3	94.1	94.1	92.4	77.4	-0.3	6.6	11.5	90.7
	Controls / Ours	Native	94.1	94.1	92.7	83.3	+0.0	0.0	11.5	95.5
		Policy-only	94.1	94.1	92.7	83.3	+0.0	0.0	11.5	95.5
		MinMandate	94.1	94.1	92.7	83.3	+0.0	0.0	11.5	95.5

Table 1: End-to-end task utility under merchant unavailability. We evaluate task success (%) $\uparrow$ and merchant adaptability across planner models, authorization conditions, and merchant-unavailability probabilities.

time. MinMandate signs merchant-selection constraints and allows the agent to choose any policy-compliant merchant at execution time. For privacy, we compare AP2, MinMandate, and a Stable Handle ablation. Native and Policy-only are reference controls, while the four planners are workload settings rather than baselines.

Utility under Merchant Unavailability

Controlled merchant outages. For each task and service class, we freeze a ranked universe of merchants satisfying the same user-approved merchant-selection constraints. We assign every task-seed-merchant tuple one deterministic availability value before execution. A merchant is unavailable at probability p_{unavail} when that value falls below p_{unavail} . The resulting masks are nested across operating points and are shared by all planners and conditions. Thus, the comparison does not depend on resampling outages or exposing different merchant failures to different protocols.

Task-scoped utility. Table 1 reports task success and failure decomposition under merchant unavailability; it measures whether payment authority can preserve utility when the preferred merchant fails. The results show that MinMandate maintains higher task success and directly supports the task-scoped payment authority claim. AP2- k loses tasks when its fixed merchant list misses an available policy-compliant

merchant. MinMandate avoids this coverage failure by selecting another policy-compliant merchant within the signed constraints; at $p_{\text{unavail}} = 0.5$, the decomposition separates market-wide outages from narrow-list failures.

Task Recovery Attack

Stable-handle linkage. Table 2 isolates the effect of a reusable public payment-layer join handle. AP2’s stable identifier enables exact cross-merchant record assembly and increases task recovery relative to its local view. MinMandate exposes no corresponding public handle, so we report its local view once instead of constructing a duplicated “joined” condition. Reintroducing a reusable handle in the Stable Handle ablation increases recovery across all evaluators. This experiment isolates public-handle linkage: MinMandate removes the reusable payment-layer join handle measured by the Stable Handle ablation, while timing, merchant, price, and deployment metadata remain declared leakage and may still support probabilistic linkage.

Overhead and Comparison

Computational practicality. Figure 3 reports setup and per-call latency for AP2 and MinMandate; it measures whether the cryptographic protection changes the workflow bottleneck. The result shows that MinMandate adds local CPU work but

Protocol Setting	Attacker View / Contrast	Evaluator			
		Human	Llama 3.3	Embedding	NLI
<i>Task recovery rate (%) ↓</i>					
AP2	Local view (one merchant)	25.0	14.3	33.9	33.9
	Public-handle join (multiple merchants)	53.1	44.6	50.0	64.3
MinMandate	Local view (one merchant)	24.1	14.3	33.9	33.9
<i>Cross-merchant attack uplift (pp) ↓</i>					
AP2	Public-handle join — Local	+28.1	+30.3	+16.1	+30.4
<i>Stable-handle ablation: attack recovery (%) ↑</i>					
MinMandate + Stable Handle	Handle-joined view (multiple merchants)	50.0	43.8	43.8	78.1

Table 2: Task recovery from payment-layer records. We report task recovery (% ↓), cross-merchant attack uplift (pp ↓), and Stable Handle ablation recovery (% ↑) across protocol settings and evaluators. In the ablation row, higher recovery means the added join handle exposes more task linkage.

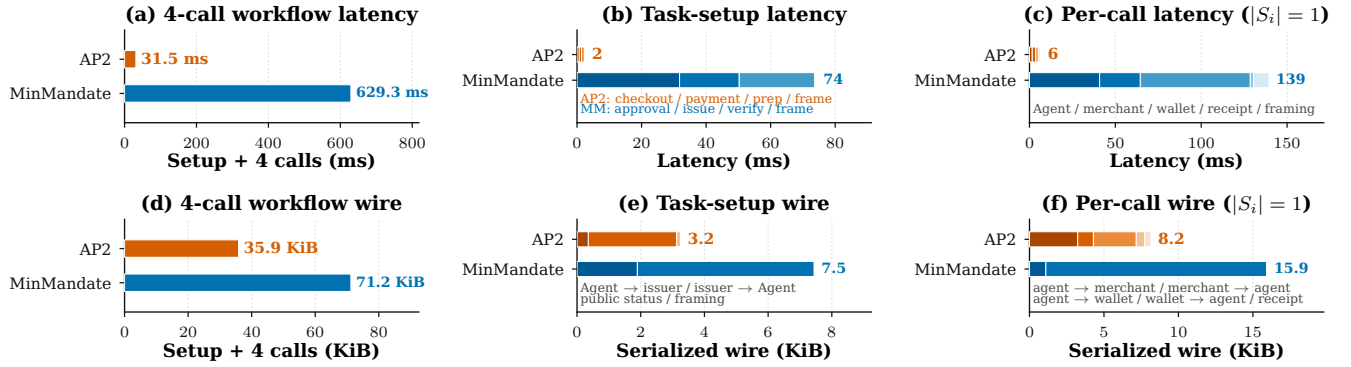


Figure 3: Runtime and communication overhead. The figure measures latency and canonical role-hop communication for AP2 and MinMandate; MinMandate adds local cryptographic cost but remains within the target four-call workflow scale.

keeps the authorization path practical, supporting the practicality claim for adaptive agentic payments. The extra work comes from issuance, issuer-hiding authorization, separate proofs contained in the service and spend views, atomic redemption, and per-call consistency binding. This raises setup and per-call latency relative to AP2, but the complete local authorization path for a four-paid-call workflow is 0.63 s (p50) on one pinned CPU. The cost is paid only when the agent makes paid calls; it adds no planner step, user interaction, or service call.

Communication practicality. Figure 3 also reports canonical role-hop bytes; it measures whether the extra verification material creates a communication bottleneck. The result shows that MinMandate uses more bytes than AP2 but remains within the paper’s target workflow scale, supporting the practicality claim for privacy-preserving paid tool calls. MinMandate sends more data because each paid call carries a service view, a spend view, view-contained proofs, serial and funding evidence, and the explicit per-call consistency binding. AP2 is cheaper on the wire because it omits these view-contained proofs and issuer-hiding authorization material. Under canonical JSON accounting, the complete four-call transcript stays on the order of 100 KiB and adds no user round trip or extra service call. The corresponding asymptotic derivations are given in Appendix B.

Conclusion

MinMandate addresses a payment gap in agentic commerce: paid workflows need adaptive merchant selection, while payment records should not let observers link calls across merchants and infer the user’s broader intent. MinMandate combines task-scoped payment authority and unlinkable per-call views: it turns one user-approved mandate into an authorization credential for choosing policy-compliant merchants within signed bounds, and derives fresh per-call views and spend evidence without introducing a stable payment identifier into external transcripts. The construction establishes authorization and spend soundness and targets external-payment-layer unlinkability under the stated honest-wallet, equal-leakage threat model. The evaluation shows higher task success under merchant unavailability and lower cross-merchant task-recovery success than AP2, supporting adaptive agentic payments with reduced payment-layer linkage.

A Protocol Specification

The construction defines the mandate, call-local anchor, spend evidence, and consistency binding. This section supplies the corresponding algorithms: the complete execution loop, the BBS credential encoding, the two presentation relations, issuer authorization, and the atomic redemption transition. All hashes take injectively encoded, domain-separated tuples.

We use i to index paid calls.

A.1 Workflow Execution Loop

The following procedure gives the complete workflow control flow. Here p_b is the planner’s scope sketch, s is its execution state, and r counts consecutive failed paid-call attempts.

Algorithm 1: Paid-agent workflow with bounded replanning

Input: Task context $\mathcal{T}_{\text{ctx}}$; tool catalog ToolCatalog; user policy UserPolicy; wallet interfaces $\mathcal{W} = (\mathcal{W}_{\text{iss}}, \mathcal{W}_{\text{red}})$; replan bound K_{replan} .

Output: $s.\text{final_answer}$ when execution completes, or $\perp$.

Mandate and credential

```

1   $p_b \leftarrow \text{LLM.PlanScope}(\mathcal{T}_{\text{ctx}}, \text{ToolCatalog}, \text{UserPolicy})$ 
2   $m \leftarrow A.\text{Compile}(\mathcal{T}_{\text{ctx}}, p_b, \text{UserPolicy})$ 
3   $\text{UAA} \leftarrow U.\text{Approve}(m)$ 
4   $z = (s_N, s_F) \xleftarrow{\$} (\mathbb{Z}_p^*)^{2L}$ 
5   $y \leftarrow (\text{H}_{\text{can}}(\text{UAA}), \{\text{Scope}_{\ell}\}_{\ell=1}^L, B, \text{expiry}, z)$ 
6   $\text{Cred} \leftarrow \mathcal{W}_{\text{iss}}.\text{Issue}(\text{UAA}, y)$ 
7  if  $\text{Cred} = \perp$  then return  $\perp$ 

```

Adaptive execution

```

8   $s \leftarrow \text{LLM.Initialize}(\mathcal{T}_{\text{ctx}}, p_b, m)$ 
9   $r \leftarrow 0$ 
10 while  $s$  is incomplete do
11    $\text{req}_i \leftarrow \text{LLM.PlanNext}(s, m)$ 
12   if  $\text{req}_i$  is free then
13     $s \leftarrow \text{LLM.Update}(s, \text{Execute}(\text{req}_i))$ ; continue

```

Paid call

Failure action: $s \leftarrow \text{LLM.Update}(s, \text{Deny}(\text{req}_i))$, $r \leftarrow r + 1$; return $\perp$ if $r > K_{\text{replan}}$, otherwise continue

```

14   $M_i \leftarrow A.\text{SelectMerchant}(\text{req}_i, m, s)$ 
15  if  $M_i = \perp$  then execute the failure action
16   $q_i \leftarrow M_i.\text{Quote}(\text{req}_i)$ 
17  if  $q_i = \perp$  then execute the failure action
18   $(\ell_i, \mathcal{S}_i) \leftarrow A.\text{SelectScopeSlots}(\text{req}_i, q_i, m, s)$ 
19  if  $(\ell_i, \mathcal{S}_i) = \perp$  then execute the failure action
20  Parse  $q_i = (a_i, \text{payee}_i, \gamma_i, \mu_i, \nu_i^q)$  and set
    $p_i = (a_i, \text{payee}_i, \text{asset}_m, \gamma_i, \mu_i, \nu_i^q, t_i)$ 
21  Sample fresh  $\eta_i, \rho_i, \alpha_i$ ; compute  $d_i^{\text{svc}}, R_i, L_i$ , the
   issuer-adapted credential, and  $(\text{Serials}_i, \text{FundTags}_i)$ 
22  Form  $(x_i^S, w_i^S)$  and  $(x_i^P, w_i^P)$  from the signed
   scope, selected slots, quote, anchor, and spend evidence

```

```

23   $\pi_i^S \leftarrow \text{BBS.Present}(\text{Cred}; x_i^S, w_i^S)$ ;
    $\pi_i^P \leftarrow \text{BBS.Present}(\text{Cred}; x_i^P, w_i^P)$ 
24   $\mathcal{V}_i^{S,0} \leftarrow (x_i^S, \pi_i^S)$ ;  $\mathcal{V}_i^{P,0} \leftarrow (x_i^P, \pi_i^P)$ 
25   $d_i^S \leftarrow \text{H}_{\text{can}}(\text{Enc}_{\text{can}}(\mathcal{V}_i^{S,0}))$ ;
    $d_i^P \leftarrow \text{H}_{\text{can}}(\text{Enc}_{\text{can}}(\mathcal{V}_i^{P,0}))$ 
26   $\text{Bind}_i \leftarrow \text{H}_{\text{bind}}(\text{Enc}_{\text{can}}((R_i, L_i), d_i^{\text{svc}}, d_i^S, d_i^P))$ 
27   $\mathcal{V}_i^S \leftarrow (\mathcal{V}_i^{S,0}, d_i^P, \text{Bind}_i)$ ;
    $\mathcal{V}_i^P \leftarrow (\mathcal{V}_i^{P,0}, d_i^S, \text{Bind}_i)$ 
28  if any view-derivation or presentation step returns  $\perp$ 
   then execute the failure action
29   $\text{ack}_i \leftarrow M_i.\text{Verify}(\text{req}_i, q_i, \mathcal{V}_i^S)$ 
30  if  $\text{ack}_i = \perp$  then execute the failure action
31   $(\text{authz}_i, \text{rcpt}_i) \leftarrow \mathcal{W}_{\text{red}}.\text{Redeem}(\mathcal{V}_i^S, \mathcal{V}_i^P, \text{ack}_i)$ 
32  if  $\text{authz}_i = \perp$  then execute the failure action
33   $r \leftarrow 0$ 
34   $\text{result}_i \leftarrow M_i.\text{Release}(\text{req}_i, \text{authz}_i)$ 
35   $s \leftarrow \text{LLM.Update}(s, (\text{result}_i, \text{rcpt}_i))$ 
36 end while

```

A.2 Task-Scoped Payment Authority

Credential encoding and issuance. The wallet interface $\mathcal{W}_{\text{iss}}.\text{Issue}(\text{UAA}, y)$ parses $\text{UAA} = (m, \sigma_U)$, verifies σ_U on the canonical encoding of m with Sig.Verify , checks that y encodes the same mandate and satisfies the capacity invariant, reserves B , and invokes the $\text{BBS.Issue}(\text{isk}_{\text{iss}}, y)$ interface. The freshness check for UAA, reservation of B , and insertion of the issued-approval marker form one linearizable issuance transaction. The wallet invokes BBS.Issue and releases the credential only after that transaction commits; a failed transaction releases no credential and does not reserve funds. The credential suite applies an injective field encoding in the order of the signed vector y ; write y_j for the resulting scalars. The BBS.Setup and BBS.KeyGen interfaces provide public generators $g_1, h_1, \dots, h_{|y|} \in \mathbb{G}_1$, $G_2 \in \mathbb{G}_2$, pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$, and the key pair with $\text{ipk}_{\text{iss}} = \text{isk}_{\text{iss}} G_2$. The concrete issuance call samples $e \in \mathbb{Z}_p$ such that $\text{isk}_{\text{iss}} + e \neq 0$, and computes

$$C(y) = g_1 + \sum_j h_j y_j, \quad \sigma_{\text{BBS}} = (A, e), \quad (10)$$

$$A = (\text{isk}_{\text{iss}} + e)^{-1} C(y).$$

The agent accepts the returned credential only if

$$e(A, \text{ipk}_{\text{iss}}) e(eA - C(y), G_2) = 1_{\mathbb{G}_T}. \quad (11)$$

It retains the state (Cred, z) : Cred contains σ_{BBS} , ipk_{iss} , and the non-seed signed coordinates, while $z = (s_N, s_F)$ contains the seed coordinates. Subsequent presentations reconstruct y in the same fixed field order, so a disclosed coordinate has the meaning of its named field in y .

A.3 Unlinkable Per-Call Views

This subsection specifies the per-call derivation from the issued credential state (Cred, z) and call inputs $(\text{req}_i, q_i, \ell_i, \mathcal{S}_i)$. Write the quote as

$$q_i = (a_i, \text{payee}_i, \gamma_i, \mu_i, \nu_i^q),$$

where the fields are the amount, payee, service class, merchant identifier, and quote nonce. The public merchant certificate Cert_{μ_i} authenticates $(\mu_i, pk_{\mu_i}, \text{service}_{\mu_i}, \gamma_{\mu_i}, \text{maxAmount}_{\mu_i}, \text{certExpiry}_{\mu_i})$, where service_{μ_i} is the certified service identifier. The spend view uses the typed payment projection. Writing asset_m for the mandate asset, it is

$$p_i = (a_i, \text{payee}_i, \text{asset}_m, \gamma_i, \mu_i, \nu_i^q, t_i), \quad (12)$$

where t_i is the call time carried by the projection. The merchant and wallet read trusted verification times τ_i^M and τ_i^W , respectively. Epoch, certificate, credential, and selected-slot validity are evaluated at these trusted times; the wallet enforces $t_i \leq \tau_i^W$. The service statement contains q_i , while the spend statement contains p_i . For a selected slot, write $\text{expiry}_{\ell_i, k} := \text{expiry}$; the mandate expiry is the signed validity bound of every slot in the credential schema. We abbreviate the disclosed spend-evidence sets as

$$\text{Serials}_i = \{N_{\ell_i, k}\}_{k \in S_i}, \quad \text{FundTags}_i = \{T_{\ell_i, k}^F\}_{k \in S_i}.$$

Issuer authorization. Issuer hiding uses Type-1 policy tags (Flamini, Friedrichs, and Lehmann 2026). For epoch ep , let $\text{ipk}_{1, \text{ep}}, \dots, \text{ipk}_{n, \text{ep}}$ be the admitted issuer keys. The policy authority samples $x_{\text{ep}} \in \mathbb{Z}_p^*$ and $G_{\text{ep}} \in \mathbb{G}_1 \setminus \{0\}$, and sets

$$T_{\text{ep}} = x_{\text{ep}}^{-1} G_{\text{ep}}, \quad \tilde{T}_{j, \text{ep}} = x_{\text{ep}}^{-1} \text{ipk}_{j, \text{ep}} \quad (j \in [n]). \quad (13)$$

The policy authority publishes an authenticated canonical epoch record P_{ep} containing the epoch, validity interval, $(G_{\text{ep}}, T_{\text{ep}})$, and the pairs $\{(\text{ipk}_{j, \text{ep}}, \tilde{T}_{j, \text{ep}})\}_{j \in [n]}$. A verifier authenticates this record and checks

$$\forall j \in [n] : \quad e(T_{\text{ep}}, \text{ipk}_{j, \text{ep}}) = e(G_{\text{ep}}, \tilde{T}_{j, \text{ep}}). \quad (14)$$

For issuance key $\text{ipk}_{\text{iss}} = \text{ipk}_{j, \text{ep}}$, the matching policy tag is the entry $\tilde{T}_{j, \text{ep}}$ associated with that key in P_{ep} . For call i , it samples $\alpha_i \in \mathbb{Z}_p^*$ and randomizes the issuer key, its policy tag, and $\sigma_{\text{BBS}} = (A, e)$ with the same scalar:

$$\begin{aligned} \text{ipk}_i^* &= \alpha_i \text{ipk}_{\text{iss}}, & \tilde{T}_i^* &= \alpha_i \tilde{T}_{j, \text{ep}}, \\ (A_i^*, e_i^*) &= (\alpha_i^{-1} A, \alpha_i e). \end{aligned} \quad (15)$$

Let $\text{Cred}_i^* = ((A_i^*, e_i^*), y)$. Its signature satisfies

$$e(A_i^*, \text{ipk}_i^*) e(e_i^* A_i^* - C(y), G_2) = 1_{\mathbb{G}_T}. \quad (16)$$

Here Cred is the long-lived credential used by the view-derivation interface. Its call-local issuer-adapted form Cred_i^* contains the same signed vector y . The combined call $\text{BBS.Present}(\text{Cred}; x, w)$ reconstructs y from (Cred, z) , applies the issuer adaptation, and proves with the resulting Cred_i^* . Both per-call proofs use this adapted signature over the original signed coordinates. Both proof statements contain $\Pi_i^{\text{auth}} = (\text{ep}, H_{\text{can}}(P_{\text{ep}}), \text{ipk}_i^*, \tilde{T}_i^*)$. After authenticating P_{ep} , each verifier resolves the record named by its hash. The merchant checks the signed epoch interval at τ_i^M , and the wallet checks it at τ_i^W . Each then checks

$$e(T_{\text{ep}}, \text{ipk}_i^*) = e(G_{\text{ep}}, \tilde{T}_i^*). \quad (17)$$

Algorithm 2: Per-call view derivation

Input: Issued credential state (Cred, z) and call inputs $(\text{req}_i, q_i, \ell_i, S_i)$.

Output: Service view $\mathcal{V}_i^S$ and spend view $\mathcal{V}_i^P$.

Statement and witnesses

- 1 Parse $q_i = (a_i, \text{payee}_i, \gamma_i, \mu_i, \nu_i^q)$; obtain public Cert_{μ_i}
- 2 $p_i \leftarrow (a_i, \text{payee}_i, \text{asset}_m, \gamma_i, \mu_i, \nu_i^q, t_i)$
- 3 Sample fresh η_i, ρ_i ; set $d_i^{\text{svc}} \leftarrow H_{\text{can}}(\text{Enc}_{\text{can}}(\text{req}_i))$
- 4 $R_i \leftarrow \text{Com}_{\text{vec}}(d_i^{\text{svc}}, q_i, H_{\text{can}}(\text{UAA}), \ell_i; \eta_i)$;
 $L_i \leftarrow \text{Com}_{\text{link}}(H_{\text{can}}(\text{UAA}), H_{\text{can}}(\text{Scope}_{\ell_i}); \rho_i)$
- 5 Authenticate the current P_{ep} and locate $\tilde{T}_{j, \text{ep}}$ for ipk_{iss}
- 6 Sample $\alpha_i \xleftarrow{\$} \mathbb{Z}_p^*$, set $\text{ipk}_i^* \leftarrow \alpha_i \text{ipk}_{\text{iss}}$, $\tilde{T}_i^* \leftarrow \alpha_i \tilde{T}_{j, \text{ep}}$,
and $(A_i^*, e_i^*) \leftarrow (\alpha_i^{-1} A, \alpha_i e)$
- 7 $\text{Cred}_i^* \leftarrow ((A_i^*, e_i^*), y)$;
 $\Pi_i^{\text{auth}} \leftarrow (\text{ep}, H_{\text{can}}(P_{\text{ep}}), \text{ipk}_i^*, \tilde{T}_i^*)$
- 8 For every $k \in S_i$, set $N_{\ell_i, k} \leftarrow s_{\ell_i}^N H_{\text{grp}}(\text{Enc}_{\text{can}}(\ell_i, k))$
and $T_{\ell_i, k}^F \leftarrow s_{\ell_i}^F H_{\text{grp}}(\text{Enc}_{\text{can}}(\ell_i, k))$
- 9 $\text{Serials}_i \leftarrow \{N_{\ell_i, k}\}_{k \in S_i}$;
 $\text{FundTags}_i \leftarrow \{T_{\ell_i, k}^F\}_{k \in S_i}$
- 10 $x_i^S \leftarrow (\Pi_i^{\text{auth}}, R_i, L_i, d_i^{\text{svc}}, q_i, \text{expiry}, \text{Cert}_{\mu_i})$
- 11 $w_i^S \leftarrow (\text{Cred}_i^*, H_{\text{can}}(\text{UAA}), \text{Scope}_{\ell_i}, \eta_i, \rho_i)$
- 12 $x_i^P \leftarrow (\Pi_i^{\text{auth}}, R_i, L_i, p_i, S_i, \text{Serials}_i, \text{FundTags}_i)$
- 13 $w_i^P \leftarrow$
 $(\text{Cred}_i^*, d_i^{\text{svc}}, q_i, H_{\text{can}}(\text{UAA}), \text{Scope}_{\ell_i}, \eta_i, \rho_i, s_{\ell_i}^N, s_{\ell_i}^F)$

Presentations and views

- 14 $\pi_i^S \leftarrow \text{BBS.Present}(\text{Cred}; x_i^S, w_i^S)$
 - 15 $\pi_i^P \leftarrow \text{BBS.Present}(\text{Cred}; x_i^P, w_i^P)$
 - 16 $\mathcal{V}_i^{S,0} \leftarrow (x_i^S, \pi_i^S)$; $\mathcal{V}_i^{P,0} \leftarrow (x_i^P, \pi_i^P)$
 - 17 $d_i^S \leftarrow H_{\text{can}}(\text{Enc}_{\text{can}}(\mathcal{V}_i^{S,0}))$;
 $d_i^P \leftarrow H_{\text{can}}(\text{Enc}_{\text{can}}(\mathcal{V}_i^{P,0}))$
 - 18 $\text{Bind}_i \leftarrow H_{\text{bind}}(\text{Enc}_{\text{can}}((R_i, L_i), d_i^{\text{svc}}, d_i^S, d_i^P))$
 - 19 $\mathcal{V}_i^S \leftarrow (\mathcal{V}_i^{S,0}, d_i^P, \text{Bind}_i)$; $\mathcal{V}_i^P \leftarrow (\mathcal{V}_i^{P,0}, d_i^S, \text{Bind}_i)$
 - 20 **return** $(\mathcal{V}_i^S, \mathcal{V}_i^P)$
-

The binding value Bind_i is a field of each completed enclosing view. Their common authorization, R_i , and L_i are proof-statement inputs; the two enclosing views share Bind_i and carry each other's pre-binding digest. Verification checks both layers as one call context.

Presentation statements. The two BBS presentations use the following public statements and hidden witnesses. This is the complete input surface of the two relation checkers. In the protocol interface, each π_i^S or π_i^P is one composite Fiat-Shamir proof object: its BBS component proves possession of the issuer-adapted credential, and its auxiliary representation components prove the call-specific equalities below under the same typed context. The security analysis separates these components only to name their distinct proof-of-knowledge and zero-knowledge assumptions; the wire protocol does not

send an unbound second proof.

Service-view relation. The deterministic checker $\text{CheckService}(x_i^S, w_i^S)$ accepts exactly when:

1. the authenticated epoch record satisfies Equation (17) for Π_i^{auth} , and Cred_i^* satisfies Equation (16) under its randomized issuer key;
2. the credential contains $\text{Scope}_{\ell_i} = (\gamma_{\ell_i}, \varphi_{\ell_i}, \{(k, c_{\ell_i, k})\})$, and its openings satisfy

$$R_i = \text{Com}_{\text{vec}}(d_i^{\text{svc}}, q_i, H_{\text{can}}(\text{UAA}), \ell_i; \eta_i),$$

$$L_i = \text{Com}_{\text{link}}(H_{\text{can}}(\text{UAA}), H_{\text{can}}(\text{Scope}_{\ell_i}); \rho_i);$$

3. Cert_{μ_i} verifies under the admission key, pk_{μ_i} is the acknowledgement key, $\text{payee}_i = \mu_i$, $\gamma_i = \gamma_{\ell_i} = \gamma_{\mu_i}$, $a_i \leq \text{maxAmount}_{\mu_i}$, and the certified record satisfies φ_{ℓ_i} ; and
4. $a_i > 0$, and expiry is the expiry coordinate signed in Cred_i^* .

$$\mathcal{R}_{\text{svc}} = \{(x, w) : \text{CheckService}(x, w) = 1\}. \quad (18)$$

Spend-view relation. The deterministic checker $\text{CheckSpend}(x_i^P, w_i^P)$ accepts iff:

1. the authenticated epoch record satisfies Equation (17) for Π_i^{auth} , and Cred_i^* satisfies Equation (16) under its randomized issuer key;
2. the credential contains one signed scope and the selected signed slots. The witness values $s_{\ell_i}^N, s_{\ell_i}^F$ equal the corresponding signed seed coordinates of Cred_i^* ;
3. p_i is canonical and has the form in Equation (12) for the hidden q_i , while asset_m matches the signed mandate; the openings satisfy the displayed equations for R_i and L_i above, and the selected indices are distinct;
4. for every $k \in S_i$,

$$N_{\ell_i, k} = s_{\ell_i}^N H_{\text{grp}}(\text{Enc}_{\text{can}}(\ell_i, k)),$$

$$T_{\ell_i, k}^F = s_{\ell_i}^F H_{\text{grp}}(\text{Enc}_{\text{can}}(\ell_i, k)),$$

$$\text{and } 0 < a_i \leq \sum_{k \in S_i} c_{\ell_i, k}.$$

$$\mathcal{R}_{\text{pay}} = \{(x, w) : \text{CheckSpend}(x, w) = 1\}. \quad (19)$$

The service proof exposes q_i ; the spend proof exposes p_i . Their openings of the same binding commitment R_i , followed by Bind_i , establish that both views describe the same call.

Merchant acknowledgement and atomic redemption. On input $(\text{req}_i, q_i, \mathcal{V}_i^S)$, merchant M_i reads τ_i^M , parses $\mathcal{V}_i^S = (\mathcal{V}_i^{S,0}, d_i^P, \text{Bind}_i)$, and accepts exactly when: (i) the displayed quote equals q_i , was issued by M_i , names M_i as the merchant, and satisfies $\text{payee}_i = \mu_i$, while Cert_{μ_i} binds pk_{μ_i} to M_i 's acknowledgement key and its certified service identifier to the service in req_i ; (ii) $d_i^{\text{svc}} = H_{\text{can}}(\text{Enc}_{\text{can}}(\text{req}_i))$; (iii) the epoch record and merchant certificate are valid at τ_i^M , $\tau_i^M \leq \text{expiry}$, and $\text{BBS.Verify}(\text{ipk}_i^*, x_i^S, \pi_i^S) = 1$ for $\mathcal{R}_{\text{svc}}$; and (iv) the digest of $\mathcal{V}_i^{S,0}$ and the cross-carried d_i^P reproduce the received Bind_i under the binding equation. It then signs $(R_i, d_i^{\text{svc}}, \text{Bind}_i)$ as ack_i . The redemption procedure below gives the corresponding $\mathcal{W}_{\text{red}}$. Redeem transition. Its persistent state is the spent-serial set Spent and the accepted-call map Accepted , keyed by the merchant and quote nonce.

Algorithm 3: $\mathcal{W}_{\text{red}}$. Redeem: atomic redemption

Input: State (Spent , Accepted); service view $\mathcal{V}_i^S$; spend view $\mathcal{V}_i^P$; acknowledgement ack_i .

Output: $(\text{auth}_i, \text{rcpt}_i)$ or $\perp$.

- 1 Read the trusted wallet time τ_i^W
 - 2 Parse (x_i^S, π_i^S) and (x_i^P, π_i^P) from the two views
 - 3 Require equal Π_i^{auth} , R_i, L_i ; recompute d_i^S from $\mathcal{V}_i^{S,0}$ and d_i^P from $\mathcal{V}_i^{P,0}$
 - 4 Require the cross-carried digests and both binding fields to equal the Bind_i computed from $(R_i, L_i), d_i^{\text{svc}}, d_i^S, d_i^P$
 - 5 Require $\text{BBS.Verify}(\text{ipk}_i^*, x_i^S, \pi_i^S) = 1$ for $\mathcal{R}_{\text{svc}}$, and $\text{BBS.Verify}(\text{ipk}_i^*, x_i^P, \pi_i^P) = 1$ for $\mathcal{R}_{\text{pay}}$
 - 6 Read q_i , expiry, Cert_{μ_i} from x_i^S , and $p_i, S_i, \text{Serials}_i, \text{FundTags}_i$ from x_i^P
 - 7 Require $t_i \leq \tau_i^W$; require the epoch record and certificate valid at τ_i^W , $\tau_i^W \leq \text{expiry}$, and $\tau_i^W \leq \text{expiry}_{\ell_i, k} = \text{expiry}$ for every $k \in S_i$
 - 8 Require Sig.Verify to accept ack_i under pk_{μ_i} on $(R_i, d_i^{\text{svc}}, \text{Bind}_i)$; **return** $\perp$ if any preceding check fails
 - 9 $d_i^{\text{acc}} \leftarrow H_{\text{can}}(\text{Enc}_{\text{can}}(\mathcal{V}_i^S, \mathcal{V}_i^P, \text{ack}_i))$
 - 10 Atomically execute the following linearizable transaction:
 - if** $\text{Accepted}[\mu_i, \nu_i^q]$ stores this digest, **return** the stored $(\text{auth}_i, \text{rcpt}_i)$
 - if** that entry is occupied or $\text{Serials}_i \cap \text{Spent} \neq \emptyset$, **return** $\perp$
 - Create auth_i and rcpt_i , each authenticating d_i^{acc}
 - Set $\text{Spent} \leftarrow \text{Spent} \cup \text{Serials}_i$ and $\text{Accepted}[\mu_i, \nu_i^q] \leftarrow (d_i^{\text{acc}}, \text{auth}_i, \text{rcpt}_i)$, then **return** $(\text{auth}_i, \text{rcpt}_i)$
-

B Complexity Analysis

We analyze the authorization and payment path after the user has approved a typed mandate. We exclude LLM planning, merchant discovery, service execution, application request and response transport, network latency, user waiting, and live payment-rail settlement. Let L be the number of service scopes, K_ℓ the number of slots in Scope_ℓ , $K = \sum_\ell K_\ell$, n_{cred} the number of signed credential coordinates, $s_i = |\mathcal{S}_i|$, and let $b_i = |\text{Enc}_{\text{can}}(\text{req}_i)|$ denote the encoded request length. Let q count paid-call executions, $\mathcal{I}_{\text{acc}}$ the freshly accepted calls, N_{iss} the issuers in the canonical epoch policy, and m_ℓ the policy-compliant merchants for scope ℓ . We use $\mathcal{V}_i^S$ and $\mathcal{V}_i^P$ for the service and spend views. The evaluated layout contributes a constant number of signed coordinates per scope and per slot, so

$$n_{\text{cred}} = n_0 + \Theta(L + K) = \Theta(L + K). \quad (20)$$

The cost of evaluating φ_ℓ is tracked separately: the evaluated exact-merchant and admitted-class predicates have fixed-size encodings and constant evaluation cost.

B.1 Time Complexity

Mandate construction and issuance. Encoding the approved mandate into the fixed credential vector and ordinary issuance

Proof	Public statement	Hidden witness
Service	$\Pi_i^{\text{auth}}, R_i, L_i, d_i^{\text{svc}}, q_i,$ $\text{expiry}, \text{Cert}_{\mu_i}$	Cred_i^* ; approval digest and signed scope; η_i, ρ_i
Spend	$\Pi_i^{\text{auth}}, R_i, L_i, p_i, S_i,$ $\{N_{\ell_i, k}, T_{\ell_i, k}^F\}_{k \in \mathcal{S}_i}$	Cred_i^* ; service digest, quote, approval digest, signed scope and slots; $\eta_i, \rho_i, s_{\ell_i}^N, s_{\ell_i}^F$

Table S3: Public statements and hidden witnesses for the two presentations.

have cost

$$T_{\text{issue}} = T_{\text{BBS.Issue}}(n_{\text{cred}}) + \Theta(n_{\text{cred}}) = \Theta(n_{\text{cred}}). \quad (21)$$

The commitment routine visits the fixed message layout once, then signing performs a constant number of scalar and group operations.

Per-call view derivation. For each paid call, the agent derives two separate presentations, one for $\mathcal{V}_i^S$ and one for $\mathcal{V}_i^P$:

$$\begin{aligned} T_{\text{derive}}(i) &= T_{\text{Present}}^S(n_{\text{cred}}) \\ &\quad + T_{\text{Present}}^P(n_{\text{cred}}, s_i) \\ &\quad + \Theta(b_i + s_i). \end{aligned} \quad (22)$$

The agent hashes the canonical request once and derives one serial and one funding tag per selected slot. BBS responses are indexed by signed-coordinate witness names; the spend presentation adds a constant number of relations and disclosed coordinates per selected slot. Thus $T_{\text{derive}}(i) = \Theta(n_{\text{cred}} + b_i + s_i)$.

Merchant verification and wallet redemption. The merchant verifies its view, the fixed admission predicate, and then signs the acknowledgement:

$$\begin{aligned} T_{\text{merchant}}(i) &= T_{\text{Verify}}^S(n_{\text{cred}}) \\ &\quad + T_{\text{AdmOK}}(\varphi_{\ell_i}) + \Theta(b_i) + O(1). \end{aligned} \quad (23)$$

Here $\Theta(b_i)$ is the recomputation of d_i^{svc} from the raw request; selected slots are not inputs to the merchant checker. The redemption wallet verifies both views and atomically checks and records all disclosed serials:

$$\begin{aligned} T_{\text{redeem}}(i) &= T_{\text{Verify}}^S(n_{\text{cred}}) \\ &\quad + T_{\text{Verify}}^P(n_{\text{cred}}, s_i) \\ &\quad + \Theta(n_{\text{cred}} + s_i) + s_i T_{\text{lookup}} + s_i T_{\text{insert}}. \end{aligned} \quad (24)$$

For the fixed predicate and an in-memory indexed spent store, the costs are $T_{\text{merchant}}(i) = \Theta(n_{\text{cred}} + b_i)$ and $T_{\text{redeem}}(i) = \Theta(n_{\text{cred}} + s_i)$. Runtime measurements use the in-memory indexed spent store specified by this bound.

Workflow, merchant universe, and issuer policy. For a warm-cache workflow with q paid-call executions,

$$\begin{aligned} T_{\text{workflow}}^{\text{warm}} &= \Theta(n_{\text{cred}}) + \sum_{i=1}^q \Theta(n_{\text{cred}} + b_i + s_i) \\ &= \Theta\left((q+1)n_{\text{cred}} + \sum_{i=1}^q (b_i + s_i)\right). \end{aligned} \quad (25)$$

The single-use bound applies only to fresh acceptances:

$$\sum_{i \in \mathcal{I}_{\text{acc}}} s_i \leq K. \quad (26)$$

Failed attempts and exact retries remain in the sum in Equation (25). AP2- k has merchant-enumeration representation $\Theta(\sum_{\ell=1}^L \min(k, m_{\ell}))$; the current fixed-size predicate profile has $A_{\text{MM}} = \Theta(L)$. This is a merchant-selection representation comparison. Issuer hiding is enabled. On an epoch refresh or issuer-material cache miss, policy validation scans and verifies all N_{iss} issuer tags, costing $\Theta(N_{\text{iss}})$, which is added once to the warm-workflow bound. Per call, the cached-policy authorization check compares fixed metadata and verifies one randomized tag, hence uses $O(1)$ pairings and does not scan the issuer registry.

B.2 Space and Communication Complexity

Agent and prover state. The agent retains (Cred, z) , signed coordinates, the coordinate map, and hidden seeds. Its persistent credential state is therefore $S_{\text{agent}} = \Theta(n_{\text{cred}}\lambda)$, rather than the constant-size BBS signature alone. One call needs prover working memory

$$S_{\text{work}}(i) = \Theta(b_i + (n_{\text{cred}} + s_i)\lambda) \quad (27)$$

bits, including the canonical request buffer and proof-generation state. The merchant needs $\Theta(b_i + n_{\text{cred}}\lambda)$ transient bits, while the wallet needs $\Theta((n_{\text{cred}} + s_i)\lambda)$.

Wallet persistent state. For freshly accepted calls, the local state satisfies

$$\begin{aligned} S_{\text{spent}} &= \Theta\left(\lambda \sum_{i \in \mathcal{I}_{\text{acc}}} s_i\right) \leq O(K\lambda), \\ S_{\text{Accepted}} &= \Theta(q_{\text{acc}}\lambda). \end{aligned} \quad (28)$$

Thus the per-credential wallet-side persistent state is $O((K + q_{\text{acc}})\lambda) = O(K\lambda)$, because every fresh acceptance consumes at least one slot. This counts the abstract local Spent and Accepted state in the redemption algorithm.

Communication. The merchant view serializes the presentation, issuer-authorization statement, link commitment, service context, quote, challenge, and certificate; the spend view additionally carries serials, funding tags, and the payment projection. The acknowledgement, settlement authorization, and receipt are separate wire objects. Selected slots occur only in the spend view. The BBS proof serializes one scalar per signed coordinate, split between disclosed messages and

responses. Under the fixed field profile,

$$\begin{aligned} C_{\text{issue}} &= \Theta(n_{\text{cred}}\lambda), \\ C_{\text{service}}(i) &= \Theta(n_{\text{cred}}\lambda), \\ C_{\text{spend}}(i) &= \Theta((n_{\text{cred}} + s_i)\lambda). \end{aligned} \quad (29)$$

The raw request is delivered separately to the merchant and is not part of either proof wire. Including issuance, both views, acknowledgements, authorizations, and receipts, the authorization/payment protocol therefore uses

$$C_{\text{workflow}} = \Theta\left(\left[(q+1)n_{\text{cred}} + \sum_{i=1}^q s_i\right]\lambda\right). \quad (30)$$

The per-call issuer-hiding statement contains a randomized issuer key and tag, not the N_{iss} -entry policy; it contributes $\Theta(\lambda)$ to each view. Distributing or refreshing the authenticated epoch record adds $\Theta(N_{\text{iss}}\lambda)$ on a cold cache and no registry-sized term to a cached call.

C Security Analysis

This appendix states the security games and reductions after the concrete protocol objects have been fixed in Appendix A.

We define separate soundness and privacy games because they protect against different coalitions. The soundness game gives the adversary the agent and any subset of admitted merchants. The privacy game keeps the agent, user, wallet, and setup, status, and merchant-admission authorities honest and gives an external coalition the colluding-merchant, payment-adapter, network, and public-log views. A single-call view-simulation lemma supplies the hybrid step for the unlinkability theorem.

Authorization and spend soundness. The soundness adversary obtains the complete agent state and corrupted-merchant signing keys and may choose and schedule all protocol messages. It wins only through a wallet-accepted redemption that violates an issued mandate, the single-use denomination budget, the service/payment correspondence, the authenticated merchant destination, or the atomic state transition. A merchant acknowledgement from a corrupted merchant is an adversarially generated input, not a claim of truthful content.

External-payment-layer privacy. The privacy adversary compares two honest-agent traces with equal declared merchant and deployment leakage. It must distinguish them using only external credential/payment transcripts and the allowed leakage. Service requests, prices, timing, merchant choices, and deployment identifiers are fixed by the leakage function rather than claimed hidden. The wallet's joined backend state and the agent's local state are outside this game.

C.1 Threat-model correspondence

Let a user task be $\mathcal{T}$. An honest agent decomposes it into paid merchant-service calls $c_1, \dots, c_n$. For privacy, one paid call is represented by a service view, an external payment-adapter projection, and a deployment record:

$$c_i = (\mathcal{V}_i^S, \text{View}_i^{\text{pay}}, \text{aux}_i), \quad \mathcal{P} = (c_1, \dots, c_n). \quad (31)$$

Here $\text{View}_i^{\text{pay}}$ is the deployment-specific external projection of the redemption and rail records; it excludes wallet backend state. The leakage function $\mathcal{L}_{\text{ext}}$ explicitly contains each merchant-visible request, service and schema, quoted price and amount, timing and order, merchant and payee choice, asset and payment metadata, OAuth, workspace, API-account, session, adapter, rail, IP-address, network, or public-log identifiers. It also contains metadata deliberately inserted by a colluding merchant and every noncryptographic public coordinate or shape exposed by the deployment-specific payment projection, including the number of selected slots, externally visible slot labels, and authorization or receipt schema fields. Thus two challenge workflows must agree on these fields whenever the deployment exposes them; the game does not try to hide a public length or schema difference. The external challenge view is

$$\text{View}_{\text{ext}}(\mathcal{P}) = \mathcal{L}_{\text{ext}}(\mathcal{P}) \cup \bigcup_{i=1}^n (\mathcal{V}_i^S \cup \text{View}_i^{\text{pay}}). \quad (32)$$

The merchant receives the raw request separately from $\mathcal{V}_i^S$; its presence in $\mathcal{L}_{\text{ext}}$ makes that visibility explicit. The common service digest, request commitment, and per-call consistency binding are call-local fields, while serials and funding tags appear only through the external payment projection when the deployment exposes them.

The trusted wallet learns the seed coordinates during issuance and may retain the joined boundary view

$$\text{Leak}_W(\mathcal{P}) = \mathcal{V}_T^{\mathcal{W}_{\text{iss}}} \cup \bigcup_{i=1}^n \mathcal{V}_i^P \cup \text{Logs}^W. \quad (33)$$

This state may directly join issuance and redemption, so it is excluded from the privacy adversary's view. Likewise, the honest agent's credential, task decomposition, slot choices, seeds, and local timing are outside the external-payment-layer privacy target.

Security games. The games use the syntax from Appendices A.2 and A.3. In soundness, the challenger maintains Issued, the issued-approval replay set, Spent, and the accepted-call map Accepted. The unified wallet, user, setup/status authorities, and merchant-admission authority remain honest. In privacy, the same parties and the agent remain honest, and no wallet or agent state is exposed. This separation lets soundness cover a malicious agent without exposing agent or wallet state in the privacy game.

Definition 1 (Authorization and spend soundness). *Experiment $\text{Exp}_{\text{MinMandate}, \mathcal{A}}^{\text{auth}}(\lambda)$ proceeds as follows.*

1. **Setup and corruption.** *The challenger runs System.Setup and initializes the persistent sets and maps. It gives $\mathcal{A}$ the public parameters, verification keys, epoch policy state, and admitted merchant records. The adversary chooses $\mathcal{M}_{\text{cor}} \subseteq \mathcal{M}_{\text{adm}}$ and receives every corrupted merchant's signing key and local state.*
2. **Issuance oracle.** *On $\mathcal{O}_{\text{issue}}(\text{UAA}, z)$, the challenger verifies the user signature, freshness of UAA, funding, the merchant-predicate syntax, and the issuance*

Phase	Time	Wire	Persistent state
Issuance	$\Theta(n_{\text{cred}})$	$\Theta(n_{\text{cred}}\lambda)$	agent: $\Theta(n_{\text{cred}}\lambda)$
View derivation	$\Theta(n_{\text{cred}} + b_i + s_i)$	service: $\Theta(n_{\text{cred}}\lambda)$; spend: $\Theta((n_{\text{cred}} + s_i)\lambda)$	–
Merchant verification	$\Theta(n_{\text{cred}} + b_i)$	acknowledgement: $O(\lambda)$	$O(1)$
Redemption	$\Theta(n_{\text{cred}} + s_i)$ expected	authorization and receipt: $O(\lambda)$	$\Theta(s_i\lambda)$ per fresh acceptance
Epoch refresh	$\Theta(N_{\text{iss}})$	$\Theta(N_{\text{iss}}\lambda)$	verifier cache: $\Theta(N_{\text{iss}}\lambda)$

Table S4: Complexity summary. Issuer-hiding authorization is enabled; the epoch-refresh row covers validation and distribution of its authenticated canonical epoch record.

Security goal	Adversary	Honest parties	Protected object	Explicit exclusions
Authorization and spend soundness	Malicious agent and admitted merchants	User, wallet, setup	Mandate-compliant redemption	Quote fairness; service truth; live-rail settlement
External-payment-layer unlinkability	Colluding merchants and external payment observers	User, agent, wallet	Additional payment-layer linkage beyond declared leakage	Service semantics; OAuth/account identity; timing; network metadata; joined wallet state

Table S5: Threat-model correspondence. The privacy goal excludes agent state, joined wallet-backend state, and declared service-side/deployment leakage.

capacity invariant. On success it issues one authorization credential Cred over the complete vector $y = (\text{H}_{\text{can}}(\text{UAA}), \{\text{Scope}_\ell\}_{\ell=1}^L, B, \text{expiry}, z)$, stores $(\text{UAA}, y, \text{Cred})$ in Issued , marks UAA issued, and returns the agent credential state (Cred, z) . Otherwise it returns reject.

- Adaptive behavior.** The adversary derives proofs locally and adaptively chooses requests, quotes, merchant acknowledgements, replay patterns, and redemption schedules. It may interleave concurrent queries and pool all agent and corrupted-merchant state. Merchant acknowledgements from honest merchants are available only after that merchant verifies the service digest, relation, and validity intervals using its trusted local clock.

- Wallet-redemption oracle.** On $\mathcal{O}_{\text{redeem}}(Q_{\text{red}})$, where

$$Q_{\text{red}} = (x^S, \pi^S, x^P, \pi^P, \text{ack}, \text{Cert}_\mu), \quad (34)$$

using its trusted local clock, the wallet independently verifies $\mathcal{R}_{\text{svc}}$ and $\mathcal{R}_{\text{pay}}$, their common issuer-authorization, anchor, and binding context, the service-side opening of the quote coordinates, the spend-side opening of the quote projection, the merchant certificate, the payment projection, the merchant acknowledgement, the epoch, certificate, credential, and selected-slot validity intervals, and distinct fresh serials. On success it atomically adds all serials to Spent , records the call-local receipt, and returns accept and a settlement authorization. Otherwise it returns reject without a state change. The challenger may tally accepted amounts for the winning condition; the protocol maintains no cumulative balance counter.

- Winning condition.** The experiment outputs 1 if any redemption is accepted without a matching entry in Issued , or

violates a signed scope record, slot entry, expiry, distinct-slot capacity, funding-tag well-formedness, serial freshness, or issuance-time funding. It also wins if both relations accept although the service quote differs from the quote projection of the wallet-visible payment projection, or if the anchor, view digests, merchant acknowledgement, certificate, and binding fail to bind the service digest, quote, merchant, payee, asset, or payment projection into one accepted call; if fresh accepted amounts under one credential exceed its signed B ; or if a settlement authorization or receipt is emitted without the same atomic transition. Quote fairness, service execution, and response truth are not winning conditions.

The advantage is

$$\text{Adv}_{\text{MinMandate}, \mathcal{A}}^{\text{auth}}(\lambda) = \Pr[\text{Exp}_{\text{MinMandate}, \mathcal{A}}^{\text{auth}}(\lambda) = 1]. \quad (35)$$

Definition 2 (External-payment-layer unlinkability). In $\text{Exp}_{\text{MinMandate}, \mathcal{A}, \mathcal{L}_{\text{ext}}}^{\text{ext-priv}}(\lambda)$, the challenger keeps the user, agent, and wallet honest. It also keeps the setup, status, and admission authorities honest. The adversary controls any colluding merchants and external payment adapters and observes the network and public logs. It may choose merchant requests, quotes, challenges, and auxiliary metadata adaptively, but the agent follows the protocol. After non-challenge honest-agent sessions, $\mathcal{A}$ submits two execution workflow specifications $\mathcal{P}_0, \mathcal{P}_1$ under the same public parameters and authenticated epoch policy, satisfying

$$\mathcal{L}_{\text{ext}}(\mathcal{P}_0) = \mathcal{L}_{\text{ext}}(\mathcal{P}_1). \quad (36)$$

The challenger samples $b \leftarrow \{0, 1\}$, honestly issues and executes $\mathcal{P}_b$, and returns $\text{View}_{\text{ext}}(\mathcal{P}_b)$ from Equation (32). It

never returns agent state or Leak_W . The adversary wins if it outputs $b' = b$. Its advantage is

$$\text{Adv}_{\text{MinMandate}, \mathcal{A}, \mathcal{L}_{\text{ext}}}^{\text{ext-priv}}(\lambda) = \left| \Pr[b' = b] - \frac{1}{2} \right|. \quad (37)$$

If Equation (36) does not hold, or if the agent or wallet is compromised, the experiment makes no privacy claim.

C.2 Authorization and Spend Soundness

We use the following primitive and system assumptions.

- User signatures, merchant-admission signatures, wallet-redemption signatures, and uncorrupted-merchant signatures are EUF-CMA secure. A merchant acknowledgement under a corrupted merchant key is adversary-generated and carries no truthfulness guarantee.
- BBS signatures are adaptively unforgeable under chosen-message attack. Derived BBS presentations are proof-of-knowledge sound and zero knowledge for presentation-hidden messages. The IHBBS_1 key-tag presentations are issuer-unlinkable against the combined issuer/verifier view in the random-oracle model and unforgeable under BBS unforgeability in the algebraic group and random-oracle models. We use the bounded multi-presentation form of these properties: the service and spend proofs for one call may share one randomized issuer key and policy tag, while every distinct call uses fresh α_i . The two same-call statements are treated as one IHBBS_1 presentation target in the reductions below.
- For each epoch, the trusted policy authority publishes the authenticated canonical record P_{ep} defined in Appendix A.3. It adds an issuer key only after the external admission and revocation checks pass. This authority is setup and admission infrastructure, not a transaction role. Agents and verifiers use the same authenticated policy state for that epoch.
- The merchant-admission authority signs the injectively encoded record

$$(\mu, pk_\mu, \text{service}_\mu, \gamma_\mu, \text{maxAmount}_\mu, \text{certExpiry}_\mu).$$

Admission attests only this binding, not merchant honesty, quote fairness, service execution, or response truth.

- The algebraic proof components used for agent-key binding, approval-and-scope-link opening equality, request redaction, request-policy satisfaction, serial derivation, and funding-tag derivation are knowledge sound and zero knowledge in the Fiat–Shamir random-oracle model.
- Scope-and-holder-link and request-vector commitments are computationally binding and hiding. The canonical hash H_{can} and the domain-separated binding hash H_{bind} are collision resistant; H_{can} is modeled as a Fiat–Shamir random oracle where algebraic proofs use it. Typed encodings are injective.
- The issuance wallet implements approval freshness, reservation of the signed budget B , and insertion of the issued-approval marker as one linearizable transaction. It releases a credential only after the reserve commits and never releases two credentials for the same signed approval.

- The $\mathcal{W}_{\text{red}}$ state machine implements the redemption-oracle transition as a linearizable atomic update of Spent and Accepted . It emits settlement authorization and a call-local receipt only after the update succeeds. The key (μ_i, ν_i^q) returns a stored receipt only for the identical accepted digest and rejects conflicting reuse; the receipt carries no stable task, mandate, issuer, funding, or wallet reference.

The following experiment formalizes the serial and funding-tag derivation used by CheckSpend .

Definition 3 (Serial-and-tag external-view pseudorandomness). Let H_{grp} be a random oracle to $\mathbb{G}_1$. The challenger samples independent hidden seeds $s_\ell^N, s_\ell^F \leftarrow \mathbb{Z}_p^*$ for each scope and gives $\mathcal{A}$ public group parameters and adaptive oracle access to H_{grp} . It accepts slot coordinates (ℓ, k) , caches the first response for each coordinate, returns that same response on repeated observations, and never reveals a challenge seed. For challenge bit b , its first response for a coordinate is

$$(N_{\ell,k}, T_{\ell,k}^F) = \begin{cases} s_\ell^N H_{\text{grp}}(\text{Enc}_{\text{can}}(\ell, k)), & b = 0, \\ s_\ell^F H_{\text{grp}}(\text{Enc}_{\text{can}}(\ell, k)) \\ (U_{\ell,k}^N, U_{\ell,k}^F), & b = 1. \\ U_{\ell,k}^N, U_{\ell,k}^F \leftarrow \mathbb{G}_1 \end{cases}$$

The adversary outputs a bit b' . Its advantage $\text{Adv}_{H_{\text{grp}}, \mathcal{A}}^{\text{stpr}}(\lambda)$ is $|\Pr[b' = b] - \frac{1}{2}|$. This instantiation-level computational assumption uses a type-3 external view with no cross-group exponent companion. Serial freshness remains an abstract predicate over disclosed serial points.

Lemma 1 (Standard-assumption basis for serial and tag privacy). If DDH is hard in $\mathbb{G}_1$, then for at most q_{stpr} distinct coordinates the advantage in Definition 3 is at most

$$2q_{\text{stpr}} \text{Adv}_{\mathbb{G}_1}^{\text{ddh}}(\lambda) + O(q_{\text{stpr}}^2/p).$$

Proof. Use one hybrid for each serial point and one for each funding-tag point. In a hybrid for coordinate (ℓ, k) , program its random-oracle base with the DDH challenge base. The DDH public seed element simulates every other point sharing the same hidden scope seed, while the challenge element is the selected real or uniform point. Independent hybrids handle s_ℓ^N and s_ℓ^F . The final term covers random-oracle base collisions and the identity point. This is a joint multi-coordinate argument; it does not replace different calls with independent per-call seeds. $\square$

Reduction loss notation. For an adversary making polynomially many oracle queries, let $q_{\text{sig}}, q_{\text{pres}}, q_{\text{rel}}, q_{\text{ih}}, q_{\text{com}}, q_{\text{can}}, q_{\text{bind}}$, and q_{stpr} denote the number of signature, presentation, auxiliary relation, IHBBS_1 , commitment, canonical-hash, binding-hash, and serial/tag targets that a reduction may have to guess. These terms are bounded by the number of issued credentials, accepted or attempted presentations, selected slots, and random-oracle queries in the experiment. The reductions below are straight-line except for the standard Fiat–Shamir knowledge extractor used for accepting algebraic proofs; the displayed bounds make the guessing loss explicit.

Soundness reduction.

Bad events. Fix the first wallet-accepted redemption that satisfies a winning condition in Definition 1. The following events partition the ways in which its verification chain can fail.

E_{sig} An honest-party signature verifies without a matching signing-oracle response.

E_{pres} An accepting BBS presentation has no extractable credential witness.

E_{rel} An accepting auxiliary relation has no extractable witness.

E_{ih} An accepting issuer-adapted credential presentation is not backed by a vector in Issued under an issuer admitted by the authenticated epoch policy.

E_{com} R_i or L_i has two inconsistent accepted openings.

E_{can} Two distinct typed canonical inputs have the same H_{can} output.

E_{bind} Two distinct call contexts have the same H_{bind} output.

A merchant acknowledgement produced with a corrupted merchant key is not included in E_{sig} : it is adversarial input and authenticates only the tuple chosen by that merchant. Let E_{fund} denote release of a credential without the successful issuance-time reserve and issued-approval update, and let E_{state} denote a settlement authorization or receipt emitted without the successful atomic update specified in the redemption oracle. By the two linearizable state-machine assumptions, $\Pr[E_{\text{fund}}] = \Pr[E_{\text{state}}] = 0$; these are system assumptions, not cryptographic reductions.

Theorem 1 (Authorization and spend soundness). *For every PPT adversary $\mathcal{A}$ in Definition 1 making polynomially many issuance and redemption queries, there exist PPT reductions $\mathcal{B}_1, \dots, \mathcal{B}_7$ against the assumptions in Appendix C: signature unforgeability, BBS presentation proof-of-knowledge soundness, algebraic relation proof-of-knowledge soundness, IHBS₁ unforgeability, commitment binding, and hash collision resistance. The loss is*

$$\begin{aligned} \text{Adv}_{\text{MM}, \mathcal{A}}^{\text{auth}} &\leq q_{\text{sig}} \text{Adv}_{\text{Sig}, \mathcal{B}_1}^{\text{euf}} + q_{\text{ih}} \text{Adv}_{\text{IHBS}_1, \mathcal{B}_2}^{\text{unf}} \\ &\quad + q_{\text{pres}} \text{Adv}_{\text{BBS}, \mathcal{B}_3}^{\text{pok}} + q_{\text{rel}} \text{Adv}_{\text{rel}, \mathcal{B}_4}^{\text{pok}} \\ &\quad + q_{\text{com}} \text{Adv}_{\text{Com}, \mathcal{B}_5}^{\text{bind}} + q_{\text{can}} \text{Adv}_{\text{H}_{\text{can}}, \mathcal{B}_6}^{\text{coll}} \\ &\quad + q_{\text{bind}} \text{Adv}_{\text{H}_{\text{bind}}, \mathcal{B}_7}^{\text{coll}}. \end{aligned} \quad (38)$$

where all terms are negligible under the assumptions above. In particular, every accepted spend is covered by signed single-use denominations, and the aggregate fresh accepted amount under one credential is at most its signed budget B .

Reduction. Step 0: select the target acceptance. The reduction simulates the public state, issuance oracle, honest-merchant interface, redemption oracle, random oracle, and persistent wallet state exactly as in Definition 1. It records every signed message, issued vector, proof statement, commitment opening supplied during honest execution, and accepted redemption. If $\mathcal{A}$ never wins, every reduction returns $\perp$. Otherwise, let Q^* be the first accepted query satisfying a winning condition. Selecting the first such query is deterministic and incurs no guessing loss.

Step 1: reduce E_{sig} to signature unforgeability ($\mathcal{B}_1$).

1. *Challenge interface.* $\mathcal{B}_1$ receives an EUF-CMA verification key and access to its signing oracle for one honest signature key used by the user, admission authority, wallet, or an uncorrupted merchant.
2. *Setup simulation.* It samples all other keys and protocol parameters honestly, embeds the challenge verification key at a uniformly selected honest signing-key position, and gives $\mathcal{A}$ the resulting public state and every corrupted-merchant secret key.
3. *Oracle handling.* A protocol query that requires the embedded secret key is encoded with the typed canonical encoder and forwarded to the signing oracle; all other signing, issuance, proof, and redemption operations are answered with known keys and honest state. Each returned signature and its exact encoded message are recorded.
4. *Embedding and abort.* If Q^* contains an unqueried valid signature under a nonselected honest key, $\mathcal{B}_1$ aborts. A signature under a corrupted merchant key is processed as adversarial input and never triggers this event.
5. *Output conversion.* If the unqueried signature in Q^* is under the embedded key, $\mathcal{B}_1$ outputs its encoded message and signature as the EUF-CMA forgery.
6. *Loss.* Guessing the relevant honest key/signature position among at most q_{sig} targets gives $\Pr[E_{\text{sig}}] \leq q_{\text{sig}} \text{Adv}_{\text{Sig}, \mathcal{B}_1}^{\text{euf}}$.

Step 2: reduce E_{ih} to IHBS₁ unforgeability ($\mathcal{B}_2$).

1. *Challenge interface.* $\mathcal{B}_2$ receives the authenticated epoch policy, admitted issuer keys, and the issuance and presentation interfaces of the bounded multi-presentation IHBS₁ unforgeability game.
2. *Setup simulation.* It embeds that policy as P_{ep} and generates all user, merchant, wallet-redemption, commitment, and auxiliary-proof parameters honestly.
3. *Oracle handling.* For each successful $\mathcal{O}_{\text{issue}}$ query, it first commits the honest funding transaction, constructs the exact signed vector $y = (H_{\text{can}}(\text{UAA}), \{\text{Scope}_\ell\}_{\ell=1}^L, B, \text{expiry}, z)$, obtains the corresponding credential from the challenge issuance interface, returns (Cred, z) , and records the issuer, vector, and credential in Issued. Failed issuance and all redemption queries are simulated honestly.
4. *Embedding and abort.* It guesses one of at most q_{ih} paired service/spend presentation targets and aborts unless Q^* uses that target. After the extractors in Steps 3–4 recover the represented vector and relations, the target event holds exactly when the accepted randomized issuer key, policy tag, and credential presentation are not backed by a recorded issued vector under an admitted base issuer.
5. *Output conversion.* It outputs the common issuer-authorization statement and the two accepting same-call presentations as an IHBS₁ unforgeability violation.
6. *Loss.* Hence $\Pr[E_{\text{ih}}] \leq q_{\text{ih}} \text{Adv}_{\text{IHBS}_1, \mathcal{B}_2}^{\text{unf}}$.

Step 3: reduce E_{pres} to BBS presentation proof-of-knowledge soundness ($\mathcal{B}_3$).

1. *Challenge interface.* $\mathcal{B}_3$ participates in the Fiat–Shamir proof-of-knowledge experiment and controls the random oracle used by the BBS presentation.

2. *Setup simulation.* It generates all protocol keys honestly and runs $\mathcal{A}$ with the public parameters and corruption state from the soundness game.
3. *Oracle handling.* It answers issuance and redemption queries honestly, records every accepting BBS presentation statement, and answers random-oracle queries consistently. For the selected presentation it invokes the standard forking extractor with the same pre-challenge transcript and a fresh Fiat–Shamir response.
4. *Embedding and abort.* It guesses one of at most q_{pres} accepting or attempted presentation targets and aborts if Q^* uses another target. Honest presentations have known witnesses and do not cause an extraction failure.
5. *Output conversion.* If the target presentation verifies but the extractor does not return a BBS credential, signed vector, and presentation randomness satisfying its relation, $\mathcal{B}_3$ outputs that transcript in the proof-of-knowledge soundness experiment.
6. *Loss.* Thus $\Pr[E_{\text{pres}}] \leq q_{\text{pres}} \text{Adv}_{\text{BBS}, \mathcal{B}_3}^{\text{pok}}$.

Step 4: reduce E_{rel} to auxiliary-relation proof-of-knowledge soundness ($\mathcal{B}_4$).

1. *Challenge interface.* $\mathcal{B}_4$ receives the public relation parameters and controls the Fiat–Shamir random oracle for the service, payment, scope, serial, and funding-tag relations.
2. *Setup simulation.* It creates all signing and commitment keys honestly and exposes the same public and corrupted state as the real game.
3. *Oracle handling.* Honest proofs are generated from known witnesses. Adversarial proof and random-oracle queries are recorded and answered consistently; the selected accepting proof is rewound with the standard extractor.
4. *Embedding and abort.* It guesses one of at most q_{rel} auxiliary-relation targets in Q^* and aborts when the guessed target is not the first accepting proof lacking a valid witness.
5. *Output conversion.* If verification succeeds but extraction fails, or returns values not satisfying the public relation, it outputs the accepting transcript as a proof-of-knowledge soundness violation.
6. *Loss.* Therefore $\Pr[E_{\text{rel}}] \leq q_{\text{rel}} \text{Adv}_{\text{rel}, \mathcal{B}_4}^{\text{pok}}$.

Conditioned on $\neg E_{\text{pres}} \wedge \neg E_{\text{rel}}$, extraction returns one credential vector, the signed budget and slot records, partial openings of R_i , an opening of L_i , and the seeds used for the disclosed serial and funding-tag points.

Step 5: reduce E_{com} to commitment binding ($\mathcal{B}_5$).

1. *Challenge interface.* $\mathcal{B}_5$ receives public commitment parameters for the request-vector or scope-and-holder-link commitment scheme.
2. *Setup simulation.* It embeds those parameters for a uniformly selected commitment family and generates all remaining parameters and keys honestly.
3. *Oracle handling.* It answers honest commitment queries with recorded messages and randomness, and uses the extracted witnesses from Steps 3–4 to recover every opening asserted by an accepting service or payment relation.

4. *Embedding and abort.* It guesses one of at most q_{com} relevant R_i or L_i targets and aborts unless the target in Q^* has two different messages with valid openings.
5. *Output conversion.* It outputs the common commitment together with the two unequal messages and their opening randomness.
6. *Loss.* Thus $\Pr[E_{\text{com}}] \leq q_{\text{com}} \text{Adv}_{\text{Com}, \mathcal{B}_5}^{\text{bind}}$.

Step 6: reduce E_{can} to collision resistance of H_{can} ($\mathcal{B}_6$).

1. *Challenge interface.* $\mathcal{B}_6$ receives the public canonical-hash description, or controls the same random oracle when H_{can} is used by Fiat–Shamir.
2. *Setup simulation.* It runs the complete soundness experiment honestly while logging every typed canonical preimage supplied to H_{can} .
3. *Oracle handling.* Hash queries are answered consistently; signing, issuance, proof, commitment, and redemption queries are handled as in the real game.
4. *Embedding and abort.* It guesses one of at most q_{can} digest comparisons in Q^* and aborts unless two semantically different service requests, payment projections, quotes, serial lists, certificates, or call contexts at that comparison yield the same digest.
5. *Output conversion.* Injective typed encoding makes the two logged preimages unequal, so $\mathcal{B}_6$ outputs them as a collision in H_{can} .
6. *Loss.* Consequently $\Pr[E_{\text{can}}] \leq q_{\text{can}} \text{Adv}_{H_{\text{can}}, \mathcal{B}_6}^{\text{coll}}$.

Step 7: reduce E_{bind} to collision resistance of H_{bind} ($\mathcal{B}_7$).

1. *Challenge interface.* $\mathcal{B}_7$ receives the domain-separated binding-hash description.
2. *Setup simulation.* It simulates the protocol honestly and records every typed tuple
$$\text{Enc}_{\text{can}}(C_i, d_i^{\text{svc}}, d_i^S, d_i^P)$$
evaluated under H_{bind} .
3. *Oracle handling.* All protocol and hash queries are answered consistently with that log.
4. *Embedding and abort.* It guesses one of at most q_{bind} accepted binding checks and aborts unless Q^* preserves Bind_i while changing at least one component of the encoded call context.
5. *Output conversion.* It outputs the two unequal encoded contexts with their common H_{bind} value.
6. *Loss.* Therefore $\Pr[E_{\text{bind}}] \leq q_{\text{bind}} \text{Adv}_{H_{\text{bind}}, \mathcal{B}_7}^{\text{coll}}$.

Step 8: derive the authorization invariant after excluding bad events. Condition on the complements of Steps 1–7 and on $\neg E_{\text{fund}} \wedge \neg E_{\text{state}}$. The extracted BBS vector is then exactly one recorded in Issued. Its approval, scope, budget B , expiry, merchant class, slot indices and capacities, and serial/funding seeds are fixed. Binding of R_i and L_i , injective typed encoding, and absence of hash collisions make the service statement, payment statement, admission record, quote, merchant acknowledgement, and call context refer to those same values. The wallet’s explicit verification therefore rules out every non-budget winning condition. This conclusion does not assert quote fairness, service execution, or response truth.

Step 9: derive the aggregate budget bound. Every accepted call discloses serials derived from its extracted signed scope/index pairs. Reusing a serial fails the same atomic transition, so the fresh accepted slot sets $\mathcal{S}_i$ under one credential are pairwise disjoint. Each relation enforces $a_i \leq \sum_{k \in \mathcal{S}_i} c_{\ell_i, k}$, while issuance enforces the final budget inequality. Hence

$$\sum_i a_i \leq \sum_i \sum_{k \in \mathcal{S}_i} c_{\ell_i, k} \leq \sum_{\ell=1}^L \sum_{k \in \mathcal{K}_\ell} c_{\ell, k} \leq B. \quad (39)$$

An aggregate-budget win therefore implies one of the bad events. Applying the union bound to the explicit losses in Steps 1–7, and using $\Pr[E_{\text{fund}}] = \Pr[E_{\text{state}}] = 0$, gives Equation (38). $\square$

Lemma 2 (Context binding and replay resistance). *Except with the hash-collision and commitment-binding probabilities in Theorem 1, an accepted transcript*

$$\tau_i^{\text{acc}} = (\Pi_i^{\text{auth}}, \pi_i^S, \pi_i^P, R_i, L_i, d_i^{\text{svc}}, \mu_i, \nu_i^q, p_i, \text{Serials}_i, \text{FundTags}_i, d_i^S, d_i^P, \text{Bind}_i, \text{ack}_i) \quad (40)$$

is bound to one redemption-verified request commitment, one disclosed payment projection, one approval-and-scope link L_i , and one set of disclosed serial points. A merchant acknowledgement, including one created by a corrupted merchant, cannot make a request redeemable without both wallet-verified relations. The call-local receipt authenticates a digest of this accepted transcript and is emitted only after the serials are atomically marked.

Proof. Step 1. Verification recomputes the issuer-authorization context, the service opening and spend-side quote projection of R_i , both interface-proof contexts, and Bind_i from typed received values.

Step 2. If the service digest, amount, payee, merchant, asset, quote nonce, proof digest, payment projection, or serial list changes while the corresponding canonical digest remains unchanged, the two unequal typed preimages give E_{can} .

Step 3. If a component digest changes while Bind_i remains unchanged, the two unequal encoded call contexts give E_{bind} .

Step 4. If the service opening or the spend-side quote projection is inconsistent with its asserted request vector, the corresponding relation is false, giving E_{rel} ; if the extracted witnesses open the quote coordinates of the same R_i to different values, they give E_{com} .

Step 5. If the approval-digest or hidden-scope coordinate changes while L_i is preserved, the extracted witnesses contain two inconsistent openings of L_i , again giving E_{com} .

Step 6. Replaying the same payment proof preserves its serial points and fails the atomic serial-freshness update. Reusing (μ_i, ν_i^q) with the identical accepted digest returns only the stored receipt; reusing it with a different digest is rejected. Emitting a new settlement authorization or receipt without the successful update gives E_{state} , while authenticating a different digest under an honest wallet key gives E_{sig} . Excluding these events establishes the claim. $\square$

C.3 External-Payment-Layer Unlinkability

Lemma 3 (Single-call external-view simulation). *Restrict Definition 2 to one paid call and supply any externally exposed serial and funding-tag points as public auxiliary input. For every PPT external adversary, all remaining fields of the real call view are simulatable from $\mathcal{L}_{\text{ext}}$ and that auxiliary input with distinguishing advantage at most*

$$\begin{aligned} \varepsilon_{\text{call}}(\lambda) \leq & q_{\text{pres}} \text{Adv}_{\text{BBS}, \mathcal{B}_1}^{\text{zk}} + q_{\text{rel}} \text{Adv}_{\text{rel}, \mathcal{B}_2}^{\text{zk}} \\ & + q_{\text{com}} \text{Adv}_{\text{com}, \mathcal{B}_3}^{\text{hide}} + q_{\text{ih}} \text{Adv}_{\text{IHBBSS}_1, \mathcal{B}_4}^{\text{unlink}} \\ & + q_{\text{can}} \text{Adv}_{\text{Hcan}, \mathcal{B}_5}^{\text{coll}} + q_{\text{bind}} \text{Adv}_{\text{Hbind}, \mathcal{B}_6}^{\text{coll}}. \end{aligned} \quad (41)$$

The statement hides only fields outside the declared merchant and deployment leakage; it is not a privacy claim against the agent or wallet. Serial and funding-tag points are replaced jointly across the full workflow in Theorem 2, preserving their shared-seed distribution until that joint hybrid.

Hybrid reduction. The simulator keeps the merchant-visible request, service/schema, amount, timing, merchant, payee, asset, service class, quote nonce, separately leaked deployment metadata, network records, and public logs fixed to $\mathcal{L}_{\text{ext}}$. Define the following hybrids, each changing one class of object.

H_0 : **real call.** Generate the complete external view exactly as in Definition 2.

$H_0 \Rightarrow H_1$: **simulate the BBS presentation.**

1. *Challenge interface.* Reduction $\mathcal{B}_1$ receives the BBS real/simulated zero-knowledge challenge for the same disclosed coordinates and public statement.
2. *Setup and oracles.* It generates all non-BBS keys honestly, answers non-challenge issuance and presentation requests through the permitted challenge oracles, and answers relation, commitment, hash, and external-metadata queries consistently.
3. *Embedding and abort.* It guesses one of at most q_{pres} presentation targets, embeds the challenge transcript in the selected call, and aborts if another target is used.
4. *Output and loss.* It outputs $\mathcal{A}$'s hybrid bit as its zero-knowledge challenge bit. Therefore $|\Pr[\mathcal{A}(H_0) = 1] - \Pr[\mathcal{A}(H_1) = 1]| \leq q_{\text{pres}} \text{Adv}_{\text{BBS}, \mathcal{B}_1}^{\text{zk}}$.

$H_1 \Rightarrow H_2$: **simulate auxiliary relation proofs.**

1. *Challenge interface.* $\mathcal{B}_2$ receives the zero-knowledge real/simulated-proof challenge for one service, payment, scope, serial, or funding-tag relation statement.
2. *Setup and oracles.* It retains the H_1 BBS simulation, generates commitments and public metadata honestly, forwards the selected proof to the challenge interface, and simulates or honestly proves all other statements using their known witnesses.
3. *Embedding and abort.* It guesses one of at most q_{rel} proof targets and aborts if the selected call uses another target. Statements remain unchanged, so the simulator never needs to alter leaked request or payment fields.
4. *Output and loss.* It forwards $\mathcal{A}$'s bit, giving $|\Pr[\mathcal{A}(H_1) = 1] - \Pr[\mathcal{A}(H_2) = 1]| \leq q_{\text{rel}} \text{Adv}_{\text{rel}, \mathcal{B}_2}^{\text{zk}}$.

$H_2 \Rightarrow H_3$: **replace hidden commitment coordinates.**

1. *Challenge interface.* $\mathcal{B}_3$ receives a commitment hiding challenge for the real hidden coordinates and equal-length dummy coordinates.
2. *Setup and oracles.* It copies every disclosed service/payment coordinate from the leakage, selects dummy values only for unopened coordinates, embeds the challenge as R_i or L_i , and recomputes all public digests and bindings from the resulting public objects. The relation proofs are already simulated in H_2 , so no inconsistent witness is required.
3. *Embedding and abort.* It guesses one of at most q_{com} commitment targets and aborts on a different target or if a purportedly hidden coordinate is required by $\mathcal{L}_{\text{ext}}$; the latter case is excluded by the lemma's leakage premise.
4. *Output and loss.* It uses $\mathcal{A}$'s bit to distinguish the commitment challenge, so $|\Pr[\mathcal{A}(H_2) = 1] - \Pr[\mathcal{A}(H_3) = 1]| \leq q_{\text{com}} \text{Adv}_{\text{Com}, \mathcal{B}_3}^{\text{hide}}$.

$H_3 \Rightarrow H_4$: **replace the issuer presentation.**

1. *Challenge interface.* $\mathcal{B}_4$ receives the bounded multi-presentation IHBS_1 issuer-unlinkability challenge between two admitted issuer credentials covered by the same authenticated canonical epoch record. The challenge contains the paired service/spend presentations that intentionally share one call-local randomized issuer key and tag.
2. *Setup and oracles.* It embeds the challenge policy key, forwards allowed authorization and presentation queries, and simulates all other protocol components as in H_3 . The disclosed policy and admission result remain fixed.
3. *Embedding and abort.* It guesses one of at most q_{ih} paired same-call targets and aborts if the selected paid call differs or if the two challenge issuers do not satisfy the same leaked policy predicate.
4. *Output and loss.* It forwards $\mathcal{A}$'s bit, yielding $|\Pr[\mathcal{A}(H_3) = 1] - \Pr[\mathcal{A}(H_4) = 1]| \leq q_{\text{ih}} \text{Adv}_{\text{IHBS}_1, \mathcal{B}_4}^{\text{unlink}}$.

$H_4 \Rightarrow H_5$: **exclude canonical-hash collisions.** $\mathcal{B}_5$ logs each typed H_{can} input while simulating H_4 and answers every hash and protocol query consistently. It guesses one of at most q_{can} comparisons and aborts unless the selected comparison contains two unequal typed inputs with the same output. On that event it outputs the pair as a collision; otherwise H_5 is H_4 conditioned on no such collision. Hence the transition costs at most $q_{\text{can}} \text{Adv}_{H_{\text{can}}, \mathcal{B}_5}^{\text{coll}}$.

$H_5 \Rightarrow H_6$: **exclude binding-hash collisions.** $\mathcal{B}_6$ analogously records every $\text{Enc}_{\text{can}}(C_i, d_i^{\text{svc}}, d_i^S, d_i^P)$ input, answers all protocol queries as in H_5 , and guesses one of at most q_{bind} binding comparisons. If two unequal inputs have one H_{bind} output, it returns them as a collision; otherwise it aborts. Thus the transition to H_6 , which is conditioned on no binding collision, costs at most $q_{\text{bind}} \text{Adv}_{H_{\text{bind}}, \mathcal{B}_6}^{\text{coll}}$.

Final simulation. In H_6 , every nonleaked credential field, proof, hidden commitment coordinate, and issuer identity is generated from public parameters, $\mathcal{L}_{\text{ext}}$, and fresh simulator randomness. Externally visible serial and funding-tag points remain the public auxiliary inputs to the single-call simulator.

All public digests and bindings are then recomputed from those simulated objects. The simulator therefore produces the complete external view without the agent or wallet state, conditional on those points. Summing the six adjacent transitions gives Equation (41). $\square$

Theorem 2 (External-payment-layer unlinkability). *Let $\mathcal{P}_0, \mathcal{P}_1$ be challenge execution workflows with equal leakage under a declared function $\mathcal{L}_{\text{ext}}$ in Definition 2. For every PPT external adversary,*

$$\begin{aligned} & \text{Adv}_{\text{MinMandate}, \mathcal{A}, \mathcal{L}_{\text{ext}}}^{\text{ext-priv}} \\ & \leq n \cdot \varepsilon_{\text{call}}(\lambda) + \text{Adv}_{H_{\text{grp}}, \mathcal{B}_{\text{st}}}^{\text{stpr}}(\lambda) \\ & \quad + q_{\text{can}}^{\text{cross}} \text{Adv}_{H_{\text{can}}}^{\text{coll}} \\ & \quad + q_{\text{bind}}^{\text{cross}} \text{Adv}_{H_{\text{bind}}}^{\text{coll}} + \varepsilon_{\text{rand}}(\lambda), \end{aligned} \quad (42)$$

where n is the number of challenge paid calls and $\varepsilon_{\text{call}}$ is the bound in Lemma 3; $q_{\text{can}}^{\text{cross}}$ and $q_{\text{bind}}^{\text{cross}}$ count only cross-call comparisons not already charged in a single-call hybrid. By Lemma 1, the serial/tag term is negligible under DDH in $\mathbb{G}_1$. The term $\varepsilon_{\text{rand}}$ bounds collisions in fresh call-local issuer, commitment, and proof randomness and is $O(n^2/p)$ for uniform scalar sampling. The theorem makes no claim when the declared leakage differs or when the agent or wallet is compromised.

Hybrid reduction. Step 1: fix the adaptive public transcript. The challenger runs every non-challenge session honestly and records the adversary's adaptive merchant requests, quotes, challenges, and metadata. At challenge time, equality of $\mathcal{L}_{\text{ext}}$ fixes the number and order of calls and all merchant-visible requests, prices, timing, merchant, asset, and service-class choices, separately leaked deployment metadata, adapter identifiers, and network/public-log records in both execution workflows. If equality fails, the game rejects before sampling b .

Step 2: define one-call hybrids. Order the n challenge paid calls after their leaked metadata has been fixed. Let G_j^b , for $j = 0, \dots, n$, be the game in which calls $1, \dots, j$ of $\mathcal{P}_b$ use the simulator from Lemma 3, while calls $j+1, \dots, n$ remain real. The simulator maintains one table for adaptive hash queries and independent per-call commitment, proof, and issuer-presentation randomness. It retains the real serial and funding-tag points, including their shared-seed correlations, as the public auxiliary inputs required by Lemma 3. It copies leaked adversarial metadata verbatim.

Step 3: reduce each adjacent transition. Suppose $\mathcal{A}$ distinguishes G_{j-1}^b from G_j^b . A reduction receives the single-call real/simulated challenge, simulates all nonchallenge sessions and the other $n-1$ challenge calls, embeds its challenge at call j , and forwards adaptive metadata and random-oracle queries consistently. It aborts only if the challenge interface rejects metadata that is not equal under $\mathcal{L}_{\text{ext}}$; that case is excluded by Step 1. It outputs $\mathcal{A}$'s bit. By Lemma 3, each transition costs at most $\varepsilon_{\text{call}}(\lambda)$, so replacing all calls costs at most $n\varepsilon_{\text{call}}(\lambda)$.

Step 4: replace serial and funding-tag points jointly. After all other call components have been simulated, one Definition 3 challenge contains every distinct (ℓ_i, k) used in

the challenge workflow. Repeated observations of an identical point remain identical, while a repeated spent coordinate is rejected by the protocol and is not a fresh challenge call. Reduction $\mathcal{B}_{\text{st}}$ embeds the complete set of challenge responses in the external payment projections and forwards $\mathcal{A}$'s bit. This single joint transition preserves the common per-scope seeds until they are replaced and costs $\text{Adv}_{H_{\text{grp}}, \mathcal{B}_{\text{st}}}^{\text{stpr}}(\lambda)$.

After this transition, issuance transcripts and wallet backend state are absent, and each external call is generated only from its leaked fields, common public parameters, and fresh simulator randomness. Equality of leakage fixes the number and externally visible shape of the point sets, so the simulator's input distributions are identical for $b = 0$ and $b = 1$. A remaining difference can occur only if unequal cross-call typed inputs collide under H_{can} or unequal cross-call contexts collide under H_{bind} .

Step 5: convert a remaining difference. A canonical-hash reduction simulates G_n^b , logs all typed preimages, guesses one of at most $q_{\text{can}}^{\text{cross}}$ cross-call comparisons, and outputs the first unequal pair with a common H_{can} value. The binding-hash reduction does the same for one of at most $q_{\text{bind}}^{\text{cross}}$ H_{bind} comparisons. If neither collision occurs, the two final views are identically distributed and the best success probability is $1/2$. Adding the n one-call losses, the joint serial/tag loss, the two cross-call collision losses, and the call-randomness collision probability gives the stated bound. $\square$

D Implementation Profile

This section specifies the encodings, credential operations, settlement records, and verifier inputs of the evaluated system.

D.1 Canonical encoding

Each mandate scope contains a signed service class and merchant-admission predicate in one of two canonical forms: an exact merchant identifier or the class predicate $\text{policy}:\text{admitted}:\gamma_\ell$. The exact form binds a call to one named merchant. The class form authorizes any merchant with a valid certificate for γ_ℓ , which is the representation used in the merchant-availability evaluation: the agent can select an available admitted merchant while the signed scope remains unchanged.

A certificate Cert_{μ_i} is accepted only when its signature verifies and its canonical record agrees on $(\mu_i, pk_{\mu_i}, \text{service}_{\mu_i}, \gamma_{\mu_i}, \text{maxAmount}_{\mu_i}, \text{certExpiry}_{\mu_i})$. Thus the $\text{AdmOK}(\varphi_\ell, \gamma_\ell, \mu_i, \text{Cert}_{\mu_i})$ predicate checks the exact service class and an admitted merchant record for a class scope, and additionally checks μ_i for equality for an exact scope. Unknown predicate versions, classes, certificate layouts, and merchant keys reject. These checks give the concrete semantics of the relations in Equations (18) and (19).

Every payment projection follows Equation (12). Its quote-shaped coordinates are bound to the hidden opening of R_i , and its asset is checked against the mandate. The settlement authorization and receipt repeat the accepted payee, asset, and amount, so all three records describe the same economic event. In the AP2 adapter, source tariffs are represented as integer nanos and converted to ISO-4217 USD minor units when the amount is exactly representable. The conversion

record travels with the matched economic trace, making the adapter's amount convention auditable alongside the protocol record. Each supported asset is registered with its minor-unit exponent and canonical encoder before issuance.

D.2 Credential implementation

The relations in Equations (10), (11), (14), (16) and (17) use BLS12-381 and SHA-256 with deterministic typed serialization and canonical compressed group/scalar encoding. Decoders reject identity points, non-canonical encodings, and points outside the prime-order subgroup. Authenticated suite and schema identifiers are domain-bound in the transcript hashes and Fiat–Shamir challenge under protocol-specific domain separation. The manifest records the identifier strings and serialized-field layout, allowing each measured transcript to be interpreted under the same ciphersuite and wire schema.

A service record carries a presentation, issuer-authorization statement, and per-call evidence; a redemption record additionally carries serials, funding tags, and the payment projection. The proof size grows with the credential coordinates represented in the presentation. Thus the transcript digests and common binding specified in A.3 are transmitted fields of the enclosing redeem request. This explicit representation lets the merchant and redemption wallet verify the same call-local binding from the records they receive.

Before online use, the system authenticates the canonical epoch record, its parameter and policy digests, strictly ordered issuer-key encodings, and every policy-membership tag. That verified immutable epoch state may be cached; a cache entry never bypasses these checks for a new policy object. In this mapping, ordinary issuance realizes Equation (10), the selective-disclosure pairing check realizes Equation (11), the randomized issuer key and tag realize Equations (14), (16) and (17), and the wire-bound Fiat–Shamir contexts realize their displayed public statements. The ciphersuite and field layout are recorded in the manifest.

D.3 Payment and redemption profile

The system canonically encodes the quote, asset, and call time in the projection of Equation (12). The enclosing view digest and per-call consistency binding cover that projection, and redemption records a call-local receipt. Merchant and wallet validity checks use their trusted local verification times. The AP2 adapter applies the same exact nanos-to-minor-unit conversion described above when producing its matched economic traces.

The redemption state machine accepts only after all typed-input, credential, proof, acknowledgement, binding, freshness, and Accepted-map checks succeed. It returns an idempotent receipt only for the identical accepted digest and rejects conflicting reuse. The resulting state transition gives each accepted call one durable receipt and makes replay, retry, and concurrent redemption behave according to the same call digest.

D.4 Verifier visibility matrix

For one paid call, the agent sees both views. The merchant sees the raw service request and the service view, then returns

Item	Merchant	Redemption wallet	Issuance
Raw service request	yes	no	no
Service view	yes	yes	no
Spend view	no	yes	no
Authorization credential body	no	no	yes
Serial/funding seeds	no	no	yes

Table S6: Verifier visibility for a paid call. The two wallet roles may share backend state while retaining the issuance and redemption interfaces shown here.

the merchant acknowledgement. The redemption wallet receives both views—including (x_i^S, π_i^S) and (x_i^P, π_i^P) —and the acknowledgement; it verifies $\mathcal{R}_{\text{svc}}$ and $\mathcal{R}_{\text{pay}}$ and accesses freshness state. The table records the remaining interface boundary.

E Evaluation Details

This section specifies the frozen inputs, comparison units, observer records, and analysis procedures that produce the reported utility, privacy, and overhead measurements.

E.1 Utility evaluation

The utility benchmark uses the 96 pre-frozen AgentDojo tasks and the four planners used in main Table 1. Each planner-condition configuration is evaluated with seeds 11, 22, and 33. Native, Policy-only, AP2- k , and MinMandate receive paired merchant-availability masks, so a condition contrast changes the authorization representation while preserving the task, planner, seed, initial state, prompt, tool schema, service response, quote, and user approved scope. This pairing isolates the utility effect of merchant coverage under the same adaptive planning episode.

The task is the analysis unit because an authorization denial affects the agent’s subsequent plan and may determine the final task outcome. The replay therefore records every paid call within its task-planner-seed episode and aggregates the episode outcome into the task-success rates shown in main Table 1. The availability replay additionally records the accepted merchant, authorization set, and availability mask for each paid call. These records separate the coverage mechanism from planner behavior. The diagnostic labels are:

```
no_available_merchant,
no_authorized_available_merchant,
class_not_authorized,
merchant_not_authorized,
payment_constraint_violation.
```

In particular, the label

```
no_authorized_available_merchant
```

identifies an episode in which the market supplies an available merchant but the condition’s authorization set omits it; this is the mechanism captured by the AP2- k comparison.

The frozen manifest binds the task, planner checkpoint, seed, initial state, prompt, tool schema, merchant catalog,

tariff, wallet/IHBBS binary, runtime topology, and analysis configuration before execution. Native, Policy-only, and MinMandate run in the same scheduler block, and complete triplets share the same task, planner, seed, and initial-state hash. The preapproved typed mandate is prepared by a deterministic mapper that reads task text and public tool policy. This design evaluates execution under a user-approved scope while preserving the same mandate input across the protocol conditions.

E.2 Privacy evaluation and observer construction

The privacy benchmark evaluates whether payment-layer records enable recovery of a workflow’s high-level intent. It uses a frozen full-wire campaign. The campaign fixes the intent-level cohort, Local and joined observer inputs, field paths, attack prompts, and the blinded bidirectional-entailment evaluation procedure before the records are scored. Holding these elements fixed makes the Local-to-joined contrast a direct measurement of the additional information provided by a reusable payment-layer field.

Actual cohort and inclusion rule. Main Table 2 uses one lexicographically selected eligible workflow for each of 96 frozen Q32 task instances, producing 118 merchant records. Each primary cell uses this same roster and a valid score from the named evaluator. The Local view contains one merchant’s prescribed observer record. The AP2 joined view assembles records using repeated public wire values, without workflow identifiers or oracle groups. MinMandate exposes no qualifying public reusable field, so main Table 2 reports its Local view once. The Stable Handle ablation supplies the corresponding handle-joined observer condition. This construction aligns the observer’s joining capability with the wire-level evidence measured in the benchmark.

Human rubric. The blinded reviewer marked a reconstruction true when it recovered the same high-level objective or decision, target or decision object, and direction of action as the original intent. Wording and operational substeps could differ. A merchant-call list was scored false when it omitted the global objective, and a reconstruction was scored false when an added claim changed the decision. The 96 blinded condition items use one shared annotation pass copied to the two score files, yielding a common human-evaluation reference across conditions.

Automatic prompts and thresholds. The intent attacker is gpt-oss:120b, temperature 0, top- $p = 1$, seed 4411, 32,768 context tokens, and at most 1,024 output tokens. It reconstructs intent from the prescribed merchant observer record. The blinded Llama judge (llama3.3:70b, temperature 0) receives the original and reconstructed intent. Its frozen prompt requires agreement on core purpose, target object, intent type, and decision direction, with no explicit high-level contradiction. The embedding judge is BAAI/bge-base-en-v1.5, with cosine threshold 0.8473754227161407. The bidirectional MiniLM NLI judge uses $\min(\text{entail}_{\rightarrow}, \text{entail}_{\leftarrow}) - \max(\text{contradict}_{\rightarrow}, \text{contradict}_{\leftarrow}) \geq -0.007030873151961714$. Both thresholds were frozen

from a disjoint 24-item synthetic development set (12 positive and 12 negative examples) before formal prediction. The three evaluators provide complementary checks: human assessment evaluates task-level semantic recovery, Llama applies the same semantic rubric in a blinded automatic setting, and embedding and NLI measure similarity and bidirectional entailment under fixed thresholds.

Reusable-field paths. The AP2 full-wire profile in main Table 2 follows AP2 v0.2. Its public reusable holder field is the JWK pair `open_checkout_payload.cnf.jwk.x` and `open_checkout_payload.cnf.jwk.y`, canonically audited as `checkout_holder_key_thumbprint`. It is present in all 118 calls, has one value per workflow, and supports cross-merchant exact assembly. The AP2 v0.2 fixture taxonomy also records `open_checkout_mandate.cnf.kid`. For x402 v2, the payer request uses `payment_payload.payload.authorization.from`. The corresponding response uses `settlement_response.payer.transaction.payload.authorization.nonce` are per-payment identifiers, not reusable workflow identifiers. These paths provide the x402 v2 field-level crosswalk.

F Additional Experiments

These experiments extend the evaluated settings.

F.1 Merchant-scale utility

Table 1 evaluates AP2 enumerations at $k \in \{1, 2, 3\}$. In the frozen catalog, 77 of 145 task-service groups contain three admitted merchants, 47 contain six, and 21 contain fifteen. To measure merchant-scale behavior under a controlled common design, the merchant-scale experiment reuses the frozen source episodes and pinned replay path but normalizes the universe to $|U| \in \{3, 5, 10, 20\}$ and replays AP2- k and MinMandate under the shared independent-unavailability mask at $p_{\text{unavail}} = 0.5$. Each planner-seed cell contains 96 tasks, and each displayed planner-level rate pools three frozen seeds (288 episodes). The normalized design measures how task success changes with the admitted merchant-universe size.

For fixed k , appended alternatives remain outside AP2’s enumerated authorization, whereas MinMandate can select across the admitted universe. The full-enumeration AP2 anchor ties MinMandate in every row, identifying merchant coverage as the source of the observed difference. The experiment uses deterministic, schema-compatible offline merchant identifiers to isolate the effect of the admitted-universe size.

F.2 Scalability

The selected-slot and credential-size sweeps measure the system at its canonical proof-object serialization boundary. Figure S4 reports their latency and serialized-size trends; each point aggregates five local runs and the shaded band spans the median to the empirical 95th percentile. The band is descriptive and is not a confidence interval.

G Additional Related Work

This appendix makes the technical positioning more explicit.

Anonymous credentials and selective disclosure. BBS, Idemix, U-Prove, and Coconut are credential primitives or anonymous-credential constructions with different issuance, presentation, and privacy profiles (Looker et al. 2026; Camenisch and Lysyanskaya 2002; Camenisch and Van Herreweghen 2002; Paquin and Zaverucha 2023; Sonnino et al. 2019). They support credential possession, attribute disclosure, and, depending on the construction and profile, randomized or selective presentations. MinMandate uses this selective-disclosure substrate, but anonymous credentials alone do not specify how a merchant-visible service statement and a wallet-visible spend statement are bound to the same paid call while avoiding a stable cross-merchant payment identifier. The protocol-level construction additionally specifies task-scoped authority, independently verifiable service and spend views, their per-call consistency binding, one-time spend evidence, merchant acknowledgement, and atomic redemption. These properties follow from the composed protocol, not BBS alone.

Privacy-preserving payments and e-cash. Private-payment and e-cash systems target payer anonymity, coin unlinkability, double-spend detection, offline payment, or transaction privacy (Chaum 1983; Chaum, Fiat, and Naor 1988; Sasson et al. 2014). The privacy game targets the additional linkage exposed to external payment-layer observers across calls in one adaptive agent workflow. Serial freshness relates to double-spend prevention, but the slots carry authorization capacity rather than anonymous coins.

Capability-based and scoped authorization. Capability and scoped-token systems delegate authority through resource, action, audience, amount, or expiry restrictions, e.g., contextual caveats and OAuth access-token scopes (Birgisson et al. 2014; Hardt 2012). Seller-, amount-, and expiry-scoped payment credentials, including AP2 and SPT, share this least-authority objective (Google 2026; Stripe 2026a). MinMandate’s task-scoped predicate supports runtime substitution among admitted merchants and is coupled to unlinkable per-call verification and single-use redemption state. This representation combines merchant selection with the policy expressiveness of scoped authorization.

Agent and tool privacy. Application-layer agent privacy and tool-security work studies prompts, service requests, outputs, OAuth identities, and tool behavior (Ruan et al. 2024; Debenedetti et al. 2024; Wang et al. 2026b,a; Shao et al. 2024; Lin et al. 2026; Zhan et al. 2025). The analysis conditions on those application-layer channels and measures payment-layer-added linkage.

Reported result	AP2 profile	Measurement role
Table 1 utility	Native SDK profile, AP2 v0.2	AP2- k enumerates approval-time merchants for task-success measurement.
Table 2 privacy	Full-wire profile, AP2 v0.2	Exact-value holder-JWK observer assembly; 96 Q32 tasks and 118 records.
Figure 3 overhead	AP2 controlled four-call, single-slot profile	Canonical role-hop latency and byte accounting.

Table S7: AP2 baseline/profile crosswalk. The table assigns a fixed AP2 v0.2 profile and measurement record to each reported result.

Planner	Condition	Task success (%) $\uparrow$			
		$ U = 3$	5	10	20
L8	AP2-1	6.6	6.6	6.6	6.6
	AP2-2	9.4	9.4	9.4	9.4
	AP2-3	11.5	11.5	11.5	11.5
	AP2- $ U $	11.5	13.2	13.9	13.9
	MinMandate	11.5	13.2	13.9	13.9
Q14	AP2-1	31.2	31.2	31.2	31.2
	AP2-2	51.4	51.4	51.4	51.4
	AP2-3	62.8	62.8	62.8	62.8
	AP2- $ U $	62.8	72.6	77.1	77.1
	MinMandate	62.8	72.6	77.1	77.1
Q32	AP2-1	30.6	30.6	30.6	30.6
	AP2-2	52.4	52.4	52.4	52.4
	AP2-3	63.9	63.9	63.9	63.9
	AP2- $ U $	63.9	73.6	78.5	78.5
	MinMandate	63.9	73.6	78.5	78.5
GPT-OSS	AP2-1	37.2	37.2	37.2	37.2
	AP2-2	61.5	61.5	61.5	61.5
	AP2-3	77.1	77.1	77.1	77.1
	AP2- $ U $	77.1	88.9	94.1	94.1
	MinMandate	77.1	88.9	94.1	94.1

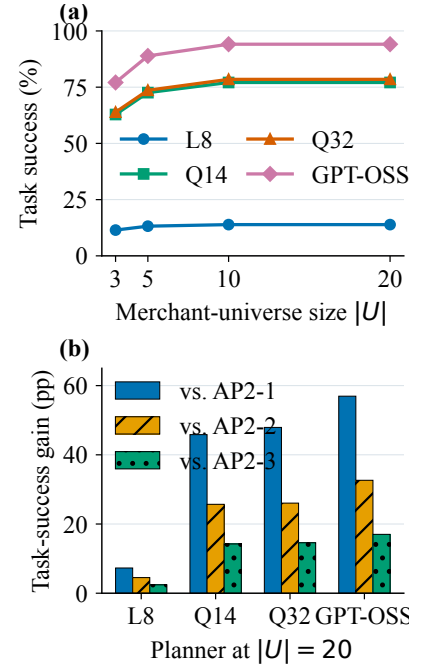


Table S8: Normalized merchant-universe extension of Table 1 at $p_{\text{unavail}} = 0.5$. Rows follow the planner-condition organization of Table 1, while columns vary the normalized merchant-universe size. The right panels show (a) MinMandate task success as the admitted universe grows and (b) its planner-specific gain over AP2- k at $|U| = 20$. AP2- $|U|$ is the diagnostic full-enumeration anchor and coincides with MinMandate.

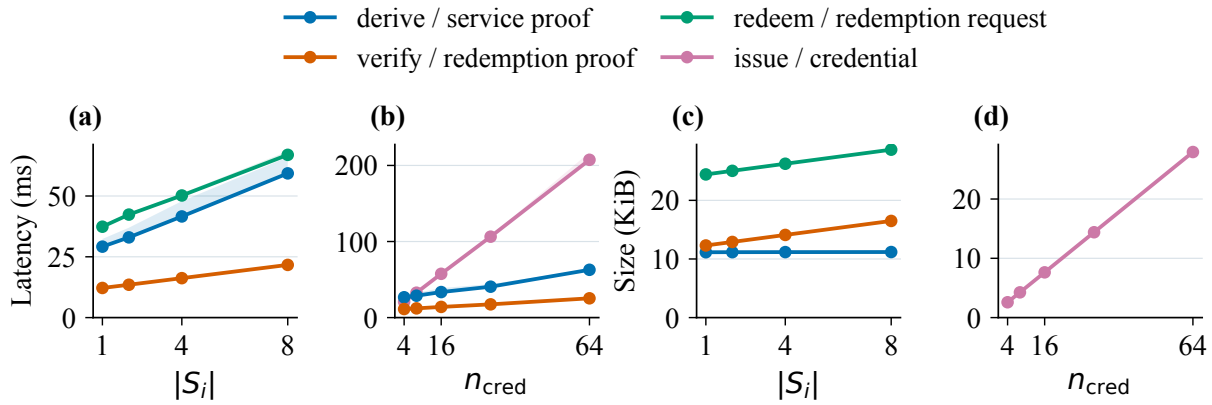


Figure S4: Scaling results. Panels (a) and (c) vary the number of selected slots in one spend proof; panels (b) and (d) vary the signed credential size with one selected slot. The four panels report latency and serialized sizes in parallel. Lines show the median and shaded bands extend to the empirical 95th percentile over five runs per point using canonical Rust proof-object serialization; the bands are descriptive, not confidence intervals.

Authorization and disclosure

System family	Primary object	Adaptive merchant choice	Verifier-specific disclosure
Scoped payment credentials	payment credential	profile-dependent	profile-dependent
Capability systems	delegated capability	profile-dependent	not intrinsic
Anonymous credentials	credential	not intrinsic	primary goal (selective-disclosure profiles)
Privacy-preserving payments/e-cash	payment or coin	profile-dependent	profile-dependent
MinMandate	task-scoped payment authority	primary goal	primary goal

Privacy and spending

System family	Cross-call payment linkage target	Spend control	Wallet privacy target
Scoped payment credentials	not intrinsic	deployment-dependent	not intrinsic
Capability systems	not intrinsic	not intrinsic	out of scope
Anonymous credentials	not intrinsic	profile-dependent	not intrinsic
Privacy-preserving payments/e-cash	primary goal (unlinkability profiles)	primary goal (e-cash profiles)	profile-dependent
MinMandate	primary goal (declared leakage)	single-use slots	out of scope

Table S9: Positioning relative to adjacent system families by primary design objectives. Entries are qualified because a family can contain profiles or deployments with additional properties.

References

- Anthropic. 2024. Introducing the Model Context Protocol. <https://www.anthropic.com/news/model-context-protocol>. Published November 25, 2024; accessed 2026-07-16.
- Attio. 2026. Attio MCP. <https://docs.attio.com/mcp/overview>. Undated documentation page; accessed 2026-07-16.
- Birgisson, A.; Politz, J. G.; Erlingsson, U.; Taly, A.; Vrable, M.; and Lentczner, M. 2014. Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud. In *Network and Distributed System Security Symposium*. Internet Society.
- Camenisch, J.; and Lysyanskaya, A. 2002. A Signature Scheme with Efficient Protocols. In *Security in Communication Networks*, 268–289. Springer.
- Camenisch, J.; and Van Herreweghen, E. 2002. Design and Implementation of the Idemix Anonymous Credential System. In *Proceedings of the 9th ACM Conference on Computer and Communications Security*, 21–30. ACM.
- Chaum, D. 1983. Blind Signatures for Untraceable Payments. In *Advances in Cryptology: Proceedings of CRYPTO 1982*, 199–203. Plenum Press.
- Chaum, D.; Fiat, A.; and Naor, M. 1988. Untraceable Electronic Cash. In *Advances in Cryptology – CRYPTO 1988*, 319–327. Springer.
- Clay. 2026a. Actions & Data Credits. <https://university.clay.com/docs/actions-data-credits>. Undated documentation page; accessed 2026-07-16.
- Clay. 2026b. MCP in Clay. <https://university.clay.com/docs/mcp-settings>. Undated documentation page; accessed 2026-07-16.
- Debenedetti, E.; Zhang, J.; Balunovic, M.; Beurer-Kellner, L.; Fischer, M.; and Tramèr, F. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems*, volume 37.
- Fett, D.; Yasuda, K.; and Campbell, B. 2025. Selective Disclosure for JSON Web Tokens. RFC 9901, <https://www.rfc-editor.org/rfc/rfc9901.html>.
- Fiat, A.; and Shamir, A. 1987. How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In *Advances in Cryptology – CRYPTO 1986*, volume 263 of *Lecture Notes in Computer Science*, 186–194. Springer.
- Flamini, A.; Friedrichs, K.; and Lehmann, A. 2026. Constant-Size Issuer Hiding for BBS Credentials via Randomizable Keys. *Cryptology ePrint Archive*, Paper 2026/369, <https://eprint.iacr.org/2026/369>.
- Goldwasser, S.; Micali, S.; and Rivest, R. L. 1988. A Digital Signature Scheme Secure Against Adaptive Chosen-Message Attacks. *SIAM Journal on Computing*, 17(2): 281–308.
- Google. 2026. Agentic Payment Protocol (AP2), Version 0.2. <https://ap2-protocol.org/ap2/specification/>. Undated specification; accessed 2026-07-16.
- Grattafiori, A.; et al. 2024. The Llama 3 Herd of Models. arXiv:2407.21783.
- Groth, J.; and Sahai, A. 2008. Efficient Non-interactive Proof Systems for Bilinear Groups. In *Advances in Cryptology – EUROCRYPT 2008*, volume 4965 of *Lecture Notes in Computer Science*, 415–432. Springer.
- Hardt, D. 2012. The OAuth 2.0 Authorization Framework. RFC 6749, <https://www.rfc-editor.org/rfc/rfc6749>. Standards Track.
- Li, Y.; Wang, L.; Wang, K.; Yang, Z.; Wang, K.; Guan, Z.; and Gao, J. 2026. A402: Binding Cryptocurrency Payments to Service Execution for Agentic Commerce. arXiv:2603.01179.
- Li, Z.; Wang, Q.; and Wang, Z. 2026. Five Attacks on x402 Agentic Payment Protocol. arXiv:2605.11781.
- Lin, Z.; Li, J.; Pan, S.; Shi, Y.; Yao, Y.; and Xu, D. 2026. Mind the Third Eye! Benchmarking Privacy Awareness in MLLM-powered Smartphone Agents. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 35626–35634.
- Ling, S.; Huang, Y.; Du, Y.; Chen, Y.; Zhou, Y.; Wu, L.; and Wang, C. 2026. Free-Riding the Agentic Web: A Systematic Security Analysis of x402 Payments. arXiv:2605.30998.
- Looker, T.; Kalos, V.; Whitehead, A.; and Lodder, M. 2026. The BBS Signature Scheme. IRTF CFRG Internet-Draft draft-irtf-cfrg-bbs-signatures-10, <https://datatracker.ietf.org/doc/draft-irtf-cfrg-bbs-signatures/>. Expired Internet-Draft, archived 2026-07-20; accessed 2026-07-29.
- Meta AI. 2024. Llama 3.3 Model Card.
- Model Context Protocol. 2025. Model Context Protocol: Schema Reference. <https://modelcontextprotocol.io/specification/2025-06-18/schema>. Accessed 2026-06-04.
- OpenAI. 2025. gpt-oss-120b & gpt-oss-20b Model Card. arXiv:2508.10925.
- Paquin, C.; and Zaverucha, G. 2023. U-Prove Cryptographic Specification V1.1 (Revision 5). Microsoft Research, <https://www.microsoft.com/en-us/research/project/u-prove/>. Official specification page; accessed 2026-07-31.
- Parikh, S.; and Surapaneni, R. 2025. Powering AI Commerce with the New Agent Payments Protocol (AP2). Google Cloud Blog, <https://cloud.google.com/blog/products/ai-machine-learning/announcing-agents-to-payments-ap2-protocol>. Published September 17, 2025; accessed 2026-07-16.
- Pedersen, T. P. 1992. Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing. In *Advances in Cryptology – CRYPTO '91*, volume 576 of *Lecture Notes in Computer Science*, 129–140. Springer.
- Reppel, E.; Caspers, R.; Leffew, K.; Organ, D.; Kim, D.; and Dalal, N. 2025. x402: An Open Standard for Internet-Native Payments. Technical report, Coinbase Developer Platform / x402. <https://www.x402.org/x402-whitepaper.pdf>; published May 6, 2025; accessed 2026-07-16.
- Ruan, Y.; Dong, H.; Wang, A.; Pitis, S.; Zhou, Y.; Ba, J.; Dubois, Y.; Maddison, C. J.; and Hashimoto, T. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. In *The Twelfth International Conference on Learning Representations*.
- Sasson, E. B.; Chiesa, A.; Garman, C.; Green, M.; Miers, I.; Tromer, E.; and Virza, M. 2014. Zerocash: Decentralized

Anonymous Payments from Bitcoin. In *IEEE Symposium on Security and Privacy*, 459–474.

Sentence Transformers. 2024. cross-encoder/nli-MiniLM2-L6-H768 Model Card.

Shao, Y.; Li, T.; Shi, W.; Liu, Y.; and Yang, D. 2024. PrivacyLens: Evaluating Privacy Norm Awareness of Language Models in Action. In *Advances in Neural Information Processing Systems*, volume 37.

Sonnino, A.; Al-Bassam, M.; Bano, S.; Meiklejohn, S.; and Danezis, G. 2019. Coconut: Threshold Issuance Selective Disclosure Credentials with Applications to Distributed Ledgers. In *Network and Distributed System Security Symposium*. Internet Society.

Stantchev, V. 2026. Hardening x402: PII-Safe Agentic Payments via Pre-Execution Metadata Filtering. arXiv:2604.11430.

Stripe. 2026a. Shared Payment Tokens. <https://docs.stripe.com/agent-commerce/concepts/shared-payment-tokens>. Private-preview documentation; accessed 2026-07-16.

Stripe. 2026b. x402 Payments. <https://docs.stripe.com/payments/machine/x402>. Accessed 2026-07-14.

Wang, Z.; Gao, Y.; Wang, Y.; Liu, S.; Sun, H.; Cheng, H.; Shi, G.; Du, H.; and Li, X. 2026a. MCPTox: A Benchmark for Tool Poisoning on Real-World MCP Servers. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 35811–35819.

Wang, Z.; Zhang, R.; Liu, Y.; Fan, W.; Jiang, W.; Zhao, Q.; Li, H.; and Xu, G. 2026b. MPMA: Preference Manipulation Attack Against Model Context Protocol. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 35838–35846.

Weinstein, J.; and Kaliski, S. 2026. Introducing the Machine Payments Protocol. Stripe Blog, <https://stripe.com/blog/machine-payments-protocol>. Published March 18, 2026; accessed 2026-07-16.

World Wide Web Consortium. 2026. Data Integrity BBS Cryptosuites v1.0. <https://www.w3.org/TR/vc-di-bbs/>. W3C Candidate Recommendation Draft, accessed 2026-06-01.

Xiao, S.; Liu, Z.; Zhang, P.; and Muennighoff, N. 2023. C-Pack: Packaged Resources To Advance General Chinese Embedding. arXiv:2309.07597.

Yang, A.; et al. 2024. Qwen2.5 Technical Report. arXiv:2412.15115.

Zhan, J.; Zhang, W.; Zhang, Z.; Xue, H.; Zhang, Y.; and Wu, Y. 2025. Portcullis: A Scalable and Verifiable Privacy Gateway for Third-Party LLM Inference. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, 1022–1030.